What is Numpy?

NumPy is the fundamental package for scientific computing in Python.

It is a Python library that provides a multidimensional array object, various derived objects (such as masked arrays and matrices), and an assortment of routines for fast operations on arrays, including mathematical, logical, shape manipulation, sorting, selecting, I/O, discrete Fourier transforms, basic linear algebra, basic statistical operations, random simulation and much more. At the core of the NumPy package, is the ndarray object. This encapsulates n-dimensional arrays of homogeneous data types

Creating Numpy array

In [1]:
```
import numpy as np
```

In [2]:
```
np.array([2,4,56,422,34,56,378,1,32]) # 1D array
```

Out[2]:
```
array([  2,   4,  56, 422,  34,  56, 378,   1,  32])
```

In [3]:
```
a = np.array([2,4,56,422,34,56,378,1,32]) # vector
print(a)
```

```
[  2   4  56 422  34  56 378   1  32]
```

In [4]:
```
type(a)
```

Out[4]:
```
numpy.ndarray
```

In [5]:
```
 # 2D Array (Matrix)
new = np.array([[ 45,34,22,2],[24,55,3,22]])
print(new)
```

```
[[45 34 22  2]
 [24 55  3 22]]
```

In [6]:
```
# 3D --- tensor
new1 = np.array([[ 45,34,22,2],[24,55,3,22],[25,56,89,75],[23,64,45,34]])
print(new1)
```

```
[[45 34 22  2]
 [24 55  3 22]
 [25 56 89 75]
 [23 64 45 34]]
```

dtype

The desired data-type for the array. If not given, then the type willbe determined as the minimum type required to hold the objects in thesequence.

In [7]:
```
np.array([11 ,23,44 ] , dtype = float)
```

Out[7]:
```
array([11., 23., 44.])
```

In [8]:
```
np.array([11 ,23,44,-21,0 ] , dtype = bool)# true for all and false for zero onl
```

Out[8]:
```
array([ True,  True,  True,  True, False])
```

In [9]:
```
np.array([11,23,44] , dtype = complex)
```

Out[9]:
```
array([11.+0.j, 23.+0.j, 44.+0.j])
```

Numpy Arrays Vs Python Sequences

NumPy arrays have a fixed size at creation, unlike Python lists (which can grow dynamically). Changing the size of an ndarray will create a new array and delete the original.

The elements in a NumPy array are all required to be of the same data type, and thus will be the same size in memory.

NumPy arrays facilitate advanced mathematical and other types of operations on large numbers of data. Typically, such operations are executed more efficiently and with less code than is possible using Python's built-in sequences.

A growing plethora of scientific and mathematical Python-based packages are using NumPy arrays; though these typically support Python-sequence input, they convert such input to NumPy arrays prior to processing, and they often output NumPy arrays.

arange

arange can be called with a varying number of positional arguments

In [10]:
```
np.arange(1,25) # 1 included , 25 last one got excluded
```

Out[10]:
```
array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16, 17,
       18, 19, 20, 21, 22, 23, 24])
```

In [11]:
```
np.arange(1,25,2) # strides --> alternate numbers
```

Out[11]:
```
array([ 1,  3,  5,  7,  9, 11, 13, 15, 17, 19, 21, 23])
```

reshape

Both of number products should be equal to umber of Items present inside the array.

In [12]:
```
np.arange(1,11).reshape(2,5) # converted 2 rows and 5 columns
```

Out[12]:
```
array([[ 1,  2,  3,  4,  5],
       [ 6,  7,  8,  9, 10]])
```

In [13]:
```
np.arange(1,15).reshape(5,2) # need 10 values but have more
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[13], line 1
----> 1 np.arange(1,15).reshape(5,2)

ValueError: cannot reshape array of size 14 into shape (5,2)
```

In [14]:
```
np.arange(1,11).reshape(5,2) # converted 2 rows and 5 columns
```

Out[14]:
```
array([[ 1,  2],
       [ 3,  4],
       [ 5,  6],
       [ 7,  8],
       [ 9, 10]])
```

In [15]:
```
np.arange(1,13).reshape(3,4) # converted 2 rows and 5 columns
```

Out[15]:
```
array([[ 1,  2,  3,  4],
       [ 5,  6,  7,  8],
       [ 9, 10, 11, 12]])
```

ones & Zeros

you can initialize the values and create values . ex: in deep learning weight shape

In [16]:
```
# np.ones and np.zeros
```

In [17]:
```
np.ones((3,4))
```

Out[17]:
```
array([[1., 1., 1., 1.],
       [1., 1., 1., 1.],
       [1., 1., 1., 1.]])
```

In [18]:
```
np.ones((3,4),dtype=int)
```

Out[18]:
```
array([[1, 1, 1, 1],
       [1, 1, 1, 1],
       [1, 1, 1, 1]])
```

In [19]:
```
np.zeros((3,4))
```

Out[19]:
```
array([[0., 0., 0., 0.],
       [0., 0., 0., 0.],
       [0., 0., 0., 0.]])
```

In [20]:
```
np.zeros((3,4),dtype=int)
```

Out[20]:
```
array([[0, 0, 0, 0],
       [0, 0, 0, 0],
       [0, 0, 0, 0]])
```

In [21]:
```
np.random.random((4,3))
```

Out[21]:
```
array([[0.57814913, 0.88383778, 0.24579712],
       [0.93365591, 0.8374746 , 0.59651916],
       [0.37985544, 0.96033935, 0.82817708],
       [0.17450838, 0.83171849, 0.86696034]])
```

linspace

It is also called as Linearly space , Linearly separable,in a given range at equal distance it creates points.

In [22]:
```
np.linspace(-10,10,10)
```

Out[22]:
```
array([-10.        ,  -7.77777778,  -5.55555556,  -3.33333333,
        -1.11111111,   1.11111111,   3.33333333,   5.55555556,
         7.77777778,  10.        ])
```

In [23]:
```
np.linspace(-10,10,5)
```

Out[23]:
```
array([-10.,  -5.,   0.,   5.,  10.])
```

In [24]:
```
np.linspace(-2,12,6)
```

Out[24]:
```
array([-2. ,  0.8,  3.6,  6.4,  9.2, 12. ])
```

identity

indentity matrix is that diagonal items will be ones and evrything will be zeros

In [25]:
```
np.identity(3)
```

Out[25]:
```
array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.]])
```

In [26]:
```
np.identity(6)
```

Out[26]:
```
array([[1., 0., 0., 0., 0., 0.],
       [0., 1., 0., 0., 0., 0.],
       [0., 0., 1., 0., 0., 0.],
       [0., 0., 0., 1., 0., 0.],
       [0., 0., 0., 0., 1., 0.],
       [0., 0., 0., 0., 0., 1.]])
```

Array Attributes

In [27]:
```
a1 = np.arange(10)
a1
```

Out[27]:
```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

In [28]:
```
a2 = np.arange(12,dtype = float)
a2
```

Out[28]:
```
array([ 0.,  1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.,  9., 10., 11.])
```

In [29]:
```
a2 = np.arange(12,dtype = float).reshape(3,4)
a2
```

Out[29]:
```
array([[ 0.,  1.,  2.,  3.],
       [ 4.,  5.,  6.,  7.],
       [ 8.,  9., 10., 11.]])
```

In [30]:
```
a3 = np.arange(8).reshape(2,2,2) # 3D tensor
a3
```

Out[30]:
```
array([[[0, 1],
        [2, 3]],

       [[4, 5],
        [6, 7]]])
```

ndim

To findout given arrays number of dimensions

In [31]:
```
a1.ndim
```

Out[31]:
```
1
```

In [32]:
```
a2.ndim
```

Out[32]:
```
2
```

In [33]:
```
a3.ndim
```

Out[33]:
```
3
```

shape

gives each item consist of no.of rows and np.of column

In [34]:
```
a1.shape # 1D array has 10 Items
```

Out[34]: (10,)

In [35]:
```
a2.shape # 3 rows and 4 columns
```

Out[35]: (3, 4)

In [36]:
```
a3.shape # first ,2 says it consists of 2D arrays .2,2 gives no.of rows and colu
```

Out[36]: (2, 2, 2)

size

gives number of items

In [37]:
```
a3
```

Out[37]:
```
array([[[0, 1],
        [2, 3]],

       [[4, 5],
        [6, 7]]])
```

In [38]:
```
a3.size
```

Out[38]: 8

In [39]:
```
a2
```

Out[39]:
```
array([[ 0.,  1.,  2.,  3.],
       [ 4.,  5.,  6.,  7.],
       [ 8.,  9., 10., 11.]])
```

In [40]:
```
a2.size
```

Out[40]: 12

In [41]:
```
a1
```

Out[41]:
```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

item size

memory occupied by the item

In [42]:
```
a1.itemsize # bytes
```

Out[42]: 4

In [43]:
```
a2.itemsize # int 64 = 8 bytes
```

Out[43]: 8

In [44]:
```
a3.itemsize # int 32 = 4 bytes
```

Out[44]: 4

dtypes

gives data type of the item

In [45]:
```
print(a1.dtype)
print(a2.dtype)
print(a3.dtype)
```

```
int32
float64
int32
```

Changing Data Type

In [46]:
```
x = np.array([33,22,2.5])
x
```

Out[46]:
```
array([33. , 22. ,  2.5])
```

In [47]:
```
x.astype(int)
```

Out[47]:
```
array([33, 22,  2])
```

Array operations

In [48]:
```
z1 = np.arange(12).reshape(3,4)
z2 = np.arange(12,24).reshape(3,4)
```

In [49]:
```
z1
```

Out[49]:
```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

In [50]:
```
z2
```

Out[50]:
```
array([[12, 13, 14, 15],
       [16, 17, 18, 19],
       [20, 21, 22, 23]])
```

scalar operations

Scalar operations on Numpy arrays include performing addition or subtraction, or multiplication on each element of a Numpy array.

arithmatic (+,-,,*,%)

In [51]:
```
z1 + 2
```

Out[51]:
```
array([[ 2,  3,  4,  5],
       [ 6,  7,  8,  9],
       [10, 11, 12, 13]])
```

In [52]:
```
z1 - 2
```

Out[52]:
```
array([[-2, -1,  0,  1],
       [ 2,  3,  4,  5],
       [ 6,  7,  8,  9]])
```

In [53]:
```
z1 * 2
```

Out[53]:
```
array([[ 0,  2,  4,  6],
       [ 8, 10, 12, 14],
       [16, 18, 20, 22]])
```

In [54]:
```
z1 ** 2
```

Out[54]:
```
array([[  0,   1,   4,   9],
       [ 16,  25,  36,  49],
       [ 64,  81, 100, 121]])
```

In [55]:
```
z1 % 2
```

Out[55]:
```
array([[0, 1, 0, 1],
       [0, 1, 0, 1],
       [0, 1, 0, 1]], dtype=int32)
```

relational Operators

The relational operators are also known as comparison operators, their main function is to return either a true or false based on the value of operands.

In [56]:
```
z2
```

Out[56]:
```
array([[12, 13, 14, 15],
       [16, 17, 18, 19],
       [20, 21, 22, 23]])
```

In [57]:
```
z2 > 2
```

Out[57]:
```
array([[ True,  True,  True,  True],
       [ True,  True,  True,  True],
       [ True,  True,  True,  True]])
```

In [58]:
```
z2 < 3
```

Out[58]:
```
array([[False, False, False, False],
       [False, False, False, False],
       [False, False, False, False]])
```

Vector Operation

We can apply on both numpy array

In [59]:
```
z1
```

Out[59]:
```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

In [60]:
```
z2
```

Out[60]:
```
array([[12, 13, 14, 15],
       [16, 17, 18, 19],
       [20, 21, 22, 23]])
```

In [61]:
```
z1 + z2
```

Out[61]:
```
array([[12, 14, 16, 18],
       [20, 22, 24, 26],
       [28, 30, 32, 34]])
```

In [62]:
```
z1 * z2
```

Out[62]:
```
array([[  0,  13,  28,  45],
       [ 64,  85, 108, 133],
       [160, 189, 220, 253]])
```

In [63]:
```
z1 - z2
```

Out[63]:
```
array([[-12, -12, -12, -12],
       [-12, -12, -12, -12],
       [-12, -12, -12, -12]])
```

In [64]:
```
z1/z2
```

Out[64]:
```
array([[0.        , 0.07692308, 0.14285714, 0.2       ],
       [0.25      , 0.29411765, 0.33333333, 0.36842105],
       [0.4       , 0.42857143, 0.45454545, 0.47826087]])
```

Array Functions

In [65]:
```
k1 = np.random.random(( 3,3 ))
k1 = np.round(k1*100)
k1
```

Out[65]:
```
array([[74., 70., 28.],
       [37., 15., 82.],
       [82., 72., 80.]])
```

In [66]:
```
np.max(k1)
```

Out[66]:
```
82.0
```

In [67]:
```
np.min(k1)
```

Out[67]:
```
15.0
```

In [68]:
```
np.sum(k1)
```

Out[68]:
```
540.0
```

In [69]:
```
np.prod(k1)
```

Out[69]: 3117675875328000.0

in numpy

0 = column ,1=row

In [70]:
```
# if we want maximum of every row
np.max(k1, axis= 1)
```

Out[70]:
```
array([74., 82., 82.])
```

In [71]:
```
# maximum of every column
np.max(k1, axis= 0)
```

Out[71]:
```
array([82., 72., 82.])
```

In [72]:
```
# product of every column
np.prod(k1, axis= 0)
```

Out[72]:
```
array([224516.,  75600., 183680.])
```

Statistics related fuctions

In [73]:
```
k1
```

Out[73]:
```
array([[74., 70., 28.],
       [37., 15., 82.],
       [82., 72., 80.]])
```

In [74]:
```
np.mean(k1)
```

Out[74]:
```
60.0
```

In [75]:
```
# mean of every column
k1.mean(axis=0)
```

Out[75]:
```
array([64.33333333, 52.33333333, 63.33333333])
```

In [76]:
```
np.median(k1)
```

Out[76]:
```
72.0
```

In [77]:
```
np.median(k1,axis =1)
```

Out[77]:
```
array([70., 37., 80.])
```

In [78]:
```
np.std(k1) # standard deviation
```

Out[78]:
```
24.463124175878363
```

In [79]:
```
np.std(k1,axis=0)
```

Out[79]:
```
array([19.60158724, 26.41127705, 24.99777768])
```

In [80]:
```
np.var(k1) # variance
```

Out[80]:
```
598.4444444444445
```

Trignometry Functions

In [81]:
```
np.sin(k1)
```

Out[81]:
```
array([[-0.98514626,  0.77389068,  0.27090579],
       [-0.64353813,  0.65028784,  0.31322878],
       [ 0.31322878,  0.25382336, -0.99388865]])
```

In [82]:
```
np.cos(k1)
```

Out[82]:
```
array([[ 0.17171734,  0.6333192 , -0.96260587],
       [ 0.76541405, -0.75968791,  0.9496777 ],
       [ 0.9496777 , -0.96725059, -0.11038724]])
```

In [83]:
```
np.tan(k1)
```

Out[83]:
```
array([[-5.73702254,  1.22195992, -0.2814296 ],
       [-0.84077126, -0.8559934 ,  0.32982641],
       [ 0.32982641, -0.26241738,  9.00365495]])
```

dot product

The numpy module of Python provides a function to perform the dot product of two arrays.

In [84]:
```
s2 = np.arange( 12 ).reshape(3,4)
s3 = np.arange( 12,24 ).reshape(4,3)
```

In [85]:
```
s2
```

Out[85]:
```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

In [86]:
```
s3
```

Out[86]:
```
array([[12, 13, 14],
       [15, 16, 17],
       [18, 19, 20],
       [21, 22, 23]])
```

In [87]:
```
np.dot(s2,s3)
```

Out[87]:
```
array([[114, 120, 126],
       [378, 400, 422],
       [642, 680, 718]])
```

Log and Exponents

In [88]:
```
np.exp(s2)
```

Out[88]:
```
array([[1.00000000e+00, 2.71828183e+00, 7.38905610e+00, 2.00855369e+01],
       [5.45981500e+01, 1.48413159e+02, 4.03428793e+02, 1.09663316e+03],
       [2.98095799e+03, 8.10308393e+03, 2.20264658e+04, 5.98741417e+04]])
```

round / floor /ceil

1. round

The numpy.round() function rounds the elements of an array to the nearest integer or to the specified number of decimals.

In [89]:
```
# Round to the nearest integer
arr = np.array([ 1.2,2.7,3.5,4.9 ])
rounded_arr = np.round(arr)
print(rounded_arr)
```

```
[1. 3. 4. 5.]
```

In [90]:
```
# Round to two decimals
arr = np.array([ 1.234 , 2.567 , 3.891 ])
rounded_arr = np.round(arr, decimals= 2 )
print(rounded_arr)
```

```
[1.23 2.57 3.89]
```

In [91]:
```
np.round(np.random.random((2,3))*100)
```

Out[91]:
```
array([[28., 92., 61.],
       [17., 17.,  2.]])
```

2 .floor

The numpy.floor() function returns the largest integer less than or equal to each element of an array.

In [92]:
```
# Floor operation
arr = np.array([ 1.2,2.7,3.5,4.9])
floored_arr = np.floor(arr)
print(floored_arr)
```

```
[1. 2. 3. 4.]
```

In [93]:
```
np.floor(np.random.random((2,3))*100)
```

Out[93]:
```
array([[68., 11., 71.],
       [ 7., 75., 92.]])
```

3. Ceil

The numpy.ceil() function returns the smallest integer greater than or equal to each element of an array.

In [94]:
```
arr = np.array([ 1.2,2.7,3.5,4.9 ])
ceiled_arr = np.ceil(arr)
```

```
print(ceiled_arr)
```

```
[2. 3. 4. 5.]
```

In [95]:
```
np.ceil(np.random.random((2,3))*100 ) # gives highest integer ex : 7.8 = 8
```

Out[95]:
```
array([[89.,  7., 82.],
       [44., 35.,  8.]])
```

Indexing and slicing

In [96]:
```
p1= np.arange(10)
p2= np.arange(12).reshape(3,4)
p3= np.arange(8).reshape(2,2,2)
```

In [97]:
```
p1
```

Out[97]:
```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

In [98]:
```
p2
```

Out[98]:
```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

In [99]:
```
p3
```

Out[99]:
```
array([[[0, 1],
        [2, 3]],

       [[4, 5],
        [6, 7]]])
```

Indexing on 1D array

In [100…
```
p1
```

Out[100…
```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

In [101…
```
p1[-1]
```

Out[101…
```
9
```

In [102…
```
p1[0]
```

Out[102…
```
0
```

In [103…
```
p1[1:4]
```

Out[103…
```
array([1, 2, 3])
```

In [104…
```
p1[3:9]
```

Out[104…
```
array([3, 4, 5, 6, 7, 8])
```

In [105…
```
p1[1:9:2]
```

Out[105…
```
array([1, 3, 5, 7])
```

indexing on 2D array

In [106…
```
p2
```

Out[106…
```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

In [107…
```
# fetching desired element : 6
p2[1,2]
```

Out[107…
```
6
```

In [108…
```
# fetching desired element : 11
p2[2,3]
```

Out[108…
```
11
```

In [109…
```
p2[0,1]
```

Out[109…
```
1
```

indexing on 3D (Tensors)

In [110…
```
p3
```

Out[110…
```
array([[[0, 1],
        [2, 3]],

       [[4, 5],
        [6, 7]]])
```

In [111…
```
# fetching desired element : 5
p3[1,0,1] # 3D array, you can think of the indices as [Layer, Row, Column].
```

Out[111…
```
5
```

EXPLANATION :Here 3D is consists of 2 ,2D array , so Firstly we take 1 because our desired is 5 is in second matrix which is 1 .and 1 row so 0 and second column so 1

In [112…
```
# fetching desired element :2
p3[0,1,0]
```

Out[112…
```
2
```

EXPLANATION :Here firstly we take 0 because our desired is 2, is in first matrix which is 0 . and 2 row so 1 and first column so 0

In [113…
```
# fetching desired element :6
p3[1,1,0]
```

Out[113…
```
6
```

EXPLANATION : Here first we take because our desired is 6, is in second matrix which is 1 . and second row so 1 and first column so 0

Slicing

Fetching Multiple items

Slicing on 1D

In [114…
```
p1
```

Out[114…
```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

In [115…
```
# fetching desired elementsare : 2,3,4
p1[2:5]
```

Out[115…
```
array([2, 3, 4])
```

In [116…
```
p1[2:5:2]
```

Out[116…
```
array([2, 4])
```

Slicing on 2D

In [117…
```
p2
```

Out[117…
```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

In [118…
```
p2[0,:] # In a 2D array, the comma separates dimensions: [row, column].
```

Out[118…
```
array([0, 1, 2, 3])
```

EXPLANATION :Here 0 represents first row and (:) represnts Total column

In [119…
```
p2[0,:]
```

Out[119…
```
array([0, 1, 2, 3])
```

In [120…
```
p2[:,2]
```

Out[120…
```
array([ 2,  6, 10])
```

EXPLANATION :Here we want all rows so (:) , and we want 3rd column so 2

In [121…
```
# fetch 5,6 and 9,10
p2
```

Out[121…
```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

In [122…
```
p2[1:3]
```

Out[122…
```
array([[ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

In [123…
```
p2[1:3,1:3]
```

Out[123…
```
array([[ 5,  6],
       [ 9, 10]])
```

EXPLANATION :Here first [1:3] we slice 2 second row is to third row is not existed which is 2 and Secondly , we take [1:3] which is same as first:we slice 2 second row is to third row is not included which is 3

In [124…
```
# fetch 0,3 and 8,11
p2
```

Out[124…
```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

In [125…
```
p2[::2,::3]
```

Out[125…
```
array([[ 0,  3],
       [ 8, 11]])
```

To fetch the corners of the first and third rows (0, 3 and 8, 11), you need to slice the rows and the columns using a step or a list.

For your specific syntax p2[::2, ::3]:

Row slice ::2: Start at the beginning and take every 2nd row. This selects the first row (index 0) and the third row (index 2).

Column slice ::3: Start at the beginning and take every 3rd column. In a row of 4 elements, this selects index 0 and index 3.

In [126…
```
# fetch 1,3 and 9,11
p2
```

Out[126…
```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

In [127…
```
p2[::2]
```

Out[127…
```
array([[ 0,  1,  2,  3],
       [ 8,  9, 10, 11]])
```

In [128…
```
p2[::2,1::2]
```

Out[128…
```
array([[ 1,  3],
       [ 9, 11]])
```

In [129…
```
# fetch only 4,7
p2
```

Out[129…
```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

In [130…
```
p2[1]
```

Out[130…
```
array([4, 5, 6, 7])
```

In [131…
```
p2[1,::3]
```

Out[131…
```
array([4, 7])
```

EXPLANATION : Here we take (1) because we want second row , second(:) for total column, (:3) jump for two steps and ignore middle ones

In [132…
```
# fetch 1,2,3 and 5,6,7
p2
```

Out[132…
```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

In [133…
```
p2[0:2]
```

Out[133…
```
array([[0, 1, 2, 3],
       [4, 5, 6, 7]])
```

In [134…
```
p2[0:2,1:]
```

Out[134…
```
array([[1, 2, 3],
       [5, 6, 7]])
```

In [135…
```
# fetch 1,3 and5,7
p2
```

Out[135…
```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

In [136…
```
p2[0:2]
```

Out[136…
```
array([[0, 1, 2, 3],
       [4, 5, 6, 7]])
```

In [137…
```
p2[0:2,1::2]
```

Out[137…
```
array([[1, 3],
       [5, 7]])
```

EXPLANATION : 0:2 selects the rows from index 0 (inclusive) to index 2 (exclusive), which means it will select the first and second rows of the array. , is used to separate row and column selections. 1::2 selects the columns starting from index 1 and selects every second column. So it will select the second and fourth columns of the array.

Slicing in 3D

In [138…
```
p3 = np.arange(27).reshape(3,3,3)
p3
```

Out[138…
```
array([[[ 0,  1,  2],
        [ 3,  4,  5],
        [ 6,  7,  8]],

       [[ 9, 10, 11],
        [12, 13, 14],
        [15, 16, 17]],

       [[18, 19, 20],
        [21, 22, 23],
        [24, 25, 26]]])
```

In [139…
```
# fetch second matrix
p3[1]
```

Out[139…
```
array([[ 9, 10, 11],
       [12, 13, 14],
       [15, 16, 17]])
```

In [140…
```
# fetch first and last
p3[::2]
```

Out[140…
```
array([[[ 0,  1,  2],
        [ 3,  4,  5],
        [ 6,  7,  8]],

       [[18, 19, 20],
        [21, 22, 23],
        [24, 25, 26]]])
```

In [141…
```
# Fetch 1 2d array's 2 row------> 3,4,5
p3
```

Out[141…
```
array([[[ 0,  1,  2],
        [ 3,  4,  5],
        [ 6,  7,  8]],

       [[ 9, 10, 11],
        [12, 13, 14],
        [15, 16, 17]],

       [[18, 19, 20],
        [21, 22, 23],
        [24, 25, 26]]])
```

In [142…
```
p3[0]
```

Out[142…
```
array([[0, 1, 2],
       [3, 4, 5],
       [6, 7, 8]])
```

In [143…
```
p3[0,1,:]
# EXPLANATION : 0 represnts first matrix , 1 represents second row , (:) means t
```

Out[143…
```
array([3, 4, 5])
```

In [144…
```
# Fetch 2 numpy array ,middle column------> 10,13,16
p3
```

Out[144…
```
array([[[ 0,  1,  2],
        [ 3,  4,  5],
        [ 6,  7,  8]],

       [[ 9, 10, 11],
        [12, 13, 14],
        [15, 16, 17]],

       [[18, 19, 20],
        [21, 22, 23],
        [24, 25, 26]]])
```

In [145…
```
p3[1]
```

Out[145…
```
array([[ 9, 10, 11],
       [12, 13, 14],
       [15, 16, 17]])
```

In [146…
```
p3[1,:,1]
#EXPLANATION : 1 respresnts middle column , (:) all columns , 1 represnts middle
```

Out[146…
```
array([10, 13, 16])
```

In [147…
```
# Fetch 3 array------>22,23,25,26
p3
```

Out[147…
```
array([[[ 0,  1,  2],
        [ 3,  4,  5],
        [ 6,  7,  8]],

       [[ 9, 10, 11],
        [12, 13, 14],
        [15, 16, 17]],

       [[18, 19, 20],
        [21, 22, 23],
        [24, 25, 26]]])
```

In [148…
```
p3[2]
```

Out[148…
```
array([[18, 19, 20],
       [21, 22, 23],
       [24, 25, 26]])
```

In [149…
```
p3[2,1:]
```

Out[149…
```
array([[21, 22, 23],
       [24, 25, 26]])
```

In [150…
```
p3[2,1:,1:]
```

Out[150…
```
array([[22, 23],
       [25, 26]])
```

EXPLANATION : Here we go through 3 stages , where 2 for last array , and (1:) from second row to total rows , and (1:) is for second column to total columns

In [151…
```
# Fetch 0, 2, 18 ,20
p3
```

Out[151…
```
array([[[ 0,  1,  2],
        [ 3,  4,  5],
        [ 6,  7,  8]],

       [[ 9, 10, 11],
        [12, 13, 14],
        [15, 16, 17]],

       [[18, 19, 20],
        [21, 22, 23],
        [24, 25, 26]]])
```

In [152…
```
p3[0::2]
```

Out[152…
```
array([[[ 0,  1,  2],
        [ 3,  4,  5],
        [ 6,  7,  8]],

       [[18, 19, 20],
        [21, 22, 23],
        [24, 25, 26]]])
```

In [153…
```
p3[0::2,0]
```

Out[153…
```
array([[ 0,  1,  2],
       [18, 19, 20]])
```

In [154…
```
p3[0::2,0,::2]
```

Out[154…
```
array([[ 0,  2],
       [18, 20]])
```

EXPLANATION : Here we take (0::2) first adn last column , so we did jump using this, and we took (0) for first row , and we (::2) ignored middle column

Iterating

In [155…
```
p1
```

Out[155…
```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

In [156…
```
# Looping on 1D array
for i in p1:
    print(i)
```

```
0
1
2
3
4
5
6
7
8
9
```

In [157…

```
p2
```

Out[157…

```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

In [158…

```
for i in p2:
    print(i)
```

```
[0 1 2 3]
[4 5 6 7]
[ 8  9 10 11]
```

In [159…

```
p3
```

Out[159…

```
array([[[ 0,  1,  2],
        [ 3,  4,  5],
        [ 6,  7,  8]],

       [[ 9, 10, 11],
        [12, 13, 14],
        [15, 16, 17]],

       [[18, 19, 20],
        [21, 22, 23],
        [24, 25, 26]]])
```

In [160…

```
for i in p3:
    print(i)
```

```
[[0 1 2]
 [3 4 5]
 [6 7 8]]
[[ 9 10 11]
 [12 13 14]
 [15 16 17]]
[[18 19 20]
 [21 22 23]
 [24 25 26]]
```

print all items in 3D using nditer ---- > first convert in to 1D and applying Loop

In [161…

```
for i in np.nditer(p3):
    print(i)
```

```
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
```

Reshaping

Transpose ---> Converts rows in to clumns ad columns into rows

In [162…
```
p2
```

Out[162…
```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

In [163…
```
np.transpose(p2)
```

Out[163…
```
array([[ 0,  4,  8],
       [ 1,  5,  9],
       [ 2,  6, 10],
       [ 3,  7, 11]])
```

In [164…
```
# another method
p2.T
```

Out[164…
```
array([[ 0,  4,  8],
       [ 1,  5,  9],
       [ 2,  6, 10],
       [ 3,  7, 11]])
```

In [165…
```
p3
```

Out[165…
```
array([[[ 0,  1,  2],
        [ 3,  4,  5],
        [ 6,  7,  8]],

       [[ 9, 10, 11],
        [12, 13, 14],
        [15, 16, 17]],

       [[18, 19, 20],
        [21, 22, 23],
        [24, 25, 26]]])
```

In [166…
```
p3.T
```

Out[166…
```
array([[[ 0,  9, 18],
        [ 3, 12, 21],
        [ 6, 15, 24]],

       [[ 1, 10, 19],
        [ 4, 13, 22],
        [ 7, 16, 25]],

       [[ 2, 11, 20],
        [ 5, 14, 23],
        [ 8, 17, 26]]])
```

Ravel

Converting any dimensions to 1D

In [167…
```
p2
```

Out[167…
```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

In [168…
```
p2.ravel()
```

Out[168…
```
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11])
```

In [169…
```
p3
```

Out[169…
```
array([[[ 0,  1,  2],
        [ 3,  4,  5],
        [ 6,  7,  8]],

       [[ 9, 10, 11],
        [12, 13, 14],
        [15, 16, 17]],

       [[18, 19, 20],
        [21, 22, 23],
        [24, 25, 26]]])
```

In [170…
```
p3.ravel()
```

Out[170…
```
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,
       17, 18, 19, 20, 21, 22, 23, 24, 25, 26])
```

Stacking

Stacking is the concept of joining arrays in NumPy. Arrays having the same dimensions can be stacked

In [171…
```
# Horizontal stacking
w1 = np.arange( 12 ).reshape(3,4)
w2 = np.arange( 12,24 ).reshape(3,4)
```

In [172…
```
w1
```

Out[172…
```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

In [173…
```
w2
```

Out[173…
```
array([[12, 13, 14, 15],
       [16, 17, 18, 19],
       [20, 21, 22, 23]])
```

In [174…
```
# using hstack for Horizontal stacking
np.hstack((w1,w2))
```

Out[174…
```
array([[ 0,  1,  2,  3, 12, 13, 14, 15],
       [ 4,  5,  6,  7, 16, 17, 18, 19],
       [ 8,  9, 10, 11, 20, 21, 22, 23]])
```

In [175…
```
# vertical stacking
np.vstack((w1,w2))
```

Out[175…
```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11],
       [12, 13, 14, 15],
       [16, 17, 18, 19],
       [20, 21, 22, 23]])
```

Splitting

its opposite of Stacking .

In [176…
```
# horizontal splitting
w1
```

Out[176…
```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

In [177…
```
np.hsplit(w1,2) # splitting by 2
```

Out[177…
```
[array([[0, 1],
        [4, 5],
        [8, 9]]),
 array([[ 2,  3],
        [ 6,  7],
        [10, 11]])]
```

In [178…
```
np.hsplit(w1,4) # splitting by 4
```

Out[178…
```
[array([[0],
        [4],
        [8]]),
 array([[1],
        [5],
        [9]]),
 array([[ 2],
        [ 6],
        [10]]),
 array([[ 3],
        [ 7],
        [11]])]
```

In [179…
```
# Vertical splitting
w2
```

Out[179…
```
array([[12, 13, 14, 15],
       [16, 17, 18, 19],
       [20, 21, 22, 23]])
```

In [180…
```
np.vsplit(w2,3) # spliting by 3 rows
```

Out[180…
```
[array([[12, 13, 14, 15]]),
 array([[16, 17, 18, 19]]),
 array([[20, 21, 22, 23]])]
```

Numpy Arrays Vs Python Sequences

NumPy arrays have a fixed size at creation, unlike Python lists (which can grow dynamically). Changing the size of an ndarray will create a new array and delete the original.

The elements in a NumPy array are all required to be of the same data type, and thus will be the same size in memory.

NumPy arrays facilitate advanced mathematical and other types of operations on large numbers of data. Typically, such operations are executed more efficiently and with less code than is possible using Python's built-in sequences.

A growing plethora of scientific and mathematical Python-based packages are using NumPy arrays; though these typically support Python-sequence input, they convert such input to NumPy arrays prior to processing, and they often output NumPy arrays

Speed of List Vs Numpy

List

In [182…
```
# Element wise addition
a = [ i for i in range (10000000)]
b = [i for i in range (10000000, 20000000)]
c = []
import time

start = time.time()
for i in range(len (a)):
    c.append(a[i] + b[i])

print(time.time()-start)
```

```
3.9885082244873047
```

numpy

In [184…
```
import numpy as np

a=np.arange(10000000)
b=np.arange(10000000 ,20000000)

start = time.time()
c=a+b
print(time.time()-start)
```

```
0.2022228240966797
```

3.9885082244873047/0.2022228240966797

so ,Numpy is Faster than Normal Python programming ,we can see in above Example.
because Numpy uses C type array

Memory Used for List Vs Numpy

List

In [188…
```
P = [i for i in range(10000000)]
import sys
sys.getsizeof(P)
```

Out[188…
```
89095160
```

Numpy

In [189…
```
R = np.arange(10000000)
sys.getsizeof(R)
```

Out[189…
```
40000112
```

In [192…
```
# we can decrease more in numpy
R = np.arange(10000000,dtype=np.int16)
sys.getsizeof(R)
```

Out[192…
```
20000112
```

Advance Indexing and Slicing

In [194…
```
# normal indexing and slicing
w = np.arange(12).reshape(4,3)
w
```

Out[194…
```
array([[ 0,  1,  2],
       [ 3,  4,  5],
       [ 6,  7,  8],
       [ 9, 10, 11]])
```

In [196…
```
# fetching 5 from array
w[1,2]
```

Out[196…
```
5
```

In [202…
```
# fetching 4,5,7,8
w[1:3]
# step 1
```

Out[202…
```
array([[3, 4, 5],
       [6, 7, 8]])
```

In [201…
```
#step 2
w[1:3,1:3]
```

Out[201…
```
array([[4, 5],
       [7, 8]])
```

Fancy Indexing

Fancy indexing allows you to select or modify specific elements based on complex conditions or combinations of indices. It provides a powerful way to manipulate array data in NumPy.

In [203…
```
w
```

Out[203…
```
array([[ 0,  1,  2],
       [ 3,  4,  5],
       [ 6,  7,  8],
       [ 9, 10, 11]])
```

In [204…
```
# fetch 1,3,4 row
w[[0,2,3]]
```

Out[204…
```
array([[ 0,  1,  2],
       [ 6,  7,  8],
       [ 9, 10, 11]])
```

In [206…
```
z= np.arange(24).reshape(6,4)
z
```

Out[206…
```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11],
       [12, 13, 14, 15],
       [16, 17, 18, 19],
       [20, 21, 22, 23]])
```

In [207…
```
# fetch 1,3,4,6 rows
z[[0,2,3,5]]
```

Out[207…
```
array([[ 0,  1,  2,  3],
       [ 8,  9, 10, 11],
       [12, 13, 14, 15],
       [20, 21, 22, 23]])
```

In [208…
```
# fetch 1,3,4 col
z[:,[0,2,3]]
```

Out[208…
```
array([[ 0,  2,  3],
       [ 4,  6,  7],
       [ 8, 10, 11],
       [12, 14, 15],
       [16, 18, 19],
       [20, 22, 23]])
```

Boolean indexing

It allows you to select elements from an array based on a Boolean condition. This allows you to extract only the elements of an array that meet a certain condition, making it easy to perform operations on specific subsets of data.

In [209…
```
G= np.random.randint(1,100,24).reshape(6,4)
G
```

Out[209…
```
array([[85, 51, 36, 71],
       [16, 70, 34,  1],
       [73, 21, 69, 78],
       [ 4, 84, 30, 95],
       [79, 55, 79, 44],
       [96, 74, 87, 40]])
```

In [211…
```
G>50
```

Out[211…
```
array([[ True,  True, False,  True],
       [False,  True, False, False],
       [ True, False,  True,  True],
       [False,  True, False,  True],
       [ True,  True,  True, False],
       [ True,  True,  True, False]])
```

In [212…
```
G[G>50]
```

Out[212…
```
array([85, 51, 71, 70, 73, 69, 78, 84, 95, 79, 55, 79, 96, 74, 87])
```

In [213…
```
# find even numbers
G % 2 ==0
```

Out[213…
```
array([[False, False,  True, False],
       [ True,  True,  True, False],
       [False, False, False,  True],
       [ True,  True,  True, False],
       [False, False, False,  True],
       [ True,  True, False,  True]])
```

In [214…
```
G[G % 2 ==0]
```

Out[214…
```
array([36, 16, 70, 34, 78,  4, 84, 30, 44, 96, 74, 40])
```

In [215…
```
(G > 50 ) & (G % 2 == 0)
```

Out[215…
```
array([[False, False, False, False],
       [False,  True, False, False],
       [False, False, False,  True],
       [False,  True, False, False],
       [False, False, False, False],
       [ True,  True, False, False]])
```

In [216…
```
G[(G > 50 ) & (G % 2 == 0)]
```

Out[216…
```
array([70, 78, 84, 96, 74])
```

In [217…
```
# find all numbers not divisible by7
G % 7 == 0
```

Out[217…
```
array([[False, False, False, False],
       [False,  True, False, False],
       [False,  True, False, False],
       [False,  True, False, False],
       [False, False, False, False],
       [False, False, False, False]])
```

In [219…
```
G[~(G % 7 == 0)] # result
```

Out[219…
```
array([85, 51, 36, 71, 16, 34,  1, 73, 69, 78,  4, 30, 95, 79, 55, 79, 44,
       96, 74, 87, 40])
```

Broadcasting

Used in Vectorization

The term broadcasting describes how NumPy treats arrays with different shapes during arithmetic operations. The smaller array is “broadcast” across the larger array so that they have compatible shapes.

In [221…
```
# same shape
a = np.arange( 6 ).reshape(2,3)
b = np.arange(6,12).reshape(2,3)
print(a)
print(b)
print(a+b)
```

```
[[0 1 2]
 [3 4 5]]
[[ 6  7  8]
 [ 9 10 11]]
[[ 6  8 10]
 [12 14 16]]
```

In [222…

```
# diff shape
a = np.arange(6).reshape(2,3)
b = np.arange(3).reshape(1,3)
print(a)
print(b)
print(a+b)
```

```
[[0 1 2]
 [3 4 5]]
[[0 1 2]]
[[0 2 4]
 [3 5 7]]
```

Broadcasting Rules

1. Make the two arrays have the same number of dimensions.

If the numbers of dimensions of the two arrays are different, add new dimensions with size 1 to the head of the array with the smaller dimension.

ex : (3,2) = 2D , (3) =1D ---> Convert into (1,3)

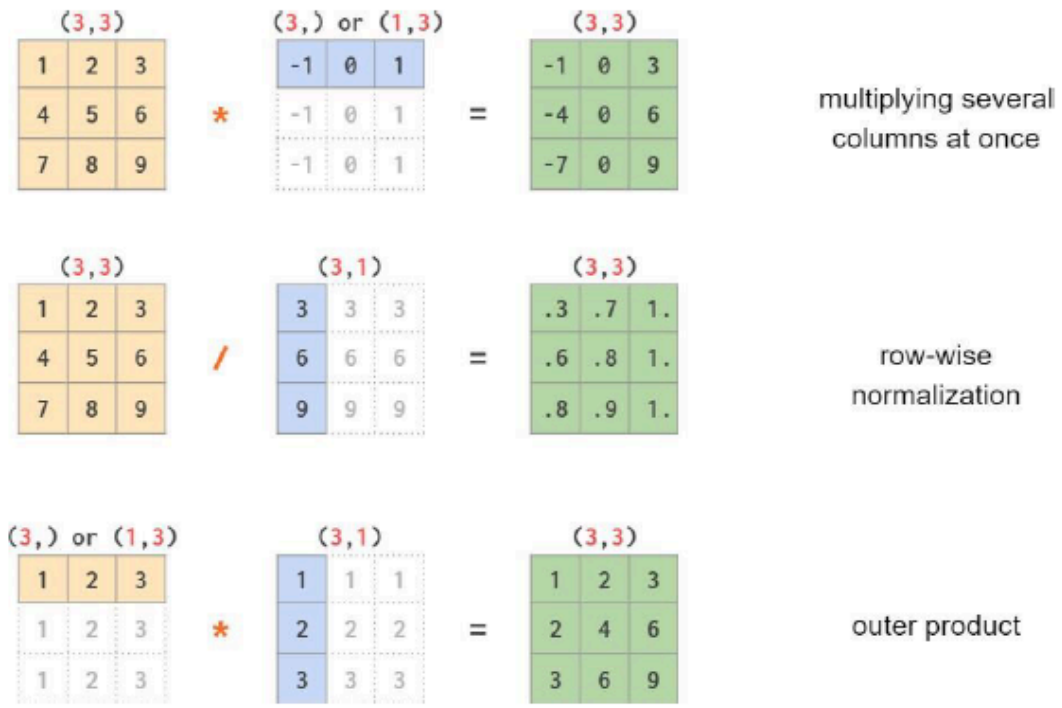
(3,3,3) = 3D ,(3) = 1D ---> Convert into (1,1,3)

2. Make each dimension of the two arrays the same size.

If the sizes of each dimension of the two arrays do not match, dimensions with size 1 are stretched to the size of the other array.

ex : (3,3)=2D ,(3) =1D ---> CONVERTED (1,3) than strech to (3,3)

If there is a dimension whose size is not 1 in either of the two arrays, it cannot be broadcasted, and an error is raised.



In [223…

```
a = np.arange(12).reshape(4,3)
b = np.arange(3)
print(a)
```

```
[[ 0  1  2]
 [ 3  4  5]
 [ 6  7  8]
 [ 9 10 11]]
```

In [224…

```
print(b)
```

```
[0 1 2]
```

In [225…

```
print(a+b)
```

```
[[ 0  2  4]
 [ 3  5  7]
 [ 6  8 10]
 [ 9 11 13]]
```

EXPLANATION : Arthematic Operation possible because , Here a = (4,3) is 2D and b =(3) is 1D so did converted (3) to (1,3) and streched to (4,3)

In [226…

```
# Could notBroadcast
a = np.arange( 12 ).reshape(3,4)
b = np.arange(3)

print(a)
print(b)
print(a+b)
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
[0 1 2]
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[226], line 7
      5 print(a)
      6 print(b)
----> 7 print(a+b)

ValueError: operands could not be broadcast together with shapes (3,4) (3,)
```

EXPLANATION : Arthematic Operation not possible because , Here a = (3,4) is 2D and b = (3) is 1D so did converted (3) to (1,3) and streched to (3,3) but , a is not equals to b . so it got failed

In [227…

```
a = np.arange(3).reshape(1,3)
b = np.arange(3).reshape(3,1)

print(a)
print(b)
print(a+b)
```

```
[[0 1 2]]
[[0]
 [1]
 [2]]
[[0 1 2]
 [1 2 3]
 [2 3 4]]
```

EXPLANATION : Arthematic Operation possible because , Here a = (1,3) is 2D and b = (3,1) is 2D so did converted (1,3) to (3,3) and b(3,1) convert (1)to 3 than (3,3) . finally it equally.

In [229…

```
a = np.arange(3).reshape(1,3)
b = np.arange(4).reshape(4,1)

print(a)
print(b)
print(a+b)
```

```
[[0 1 2]]
[[0]
 [1]
 [2]
 [3]]
[[0 1 2]
 [1 2 3]
 [2 3 4]
 [3 4 5]]
```

EXPLANATION : Same as before

In [230…

```
a = np.array([1])
b = np.arange(4).reshape(2,2)

print(a)
print(b)
print(a+b)
```

```
[1]
[[0 1]
 [2 3]]
[[1 2]
 [3 4]]
```

In [231…

```
# doesnt work
a = np.arange(12).reshape(3,4)
b = np.arange(12).reshape(4,3)

print(a)
print(b)
print(a+b)
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
[[ 0  1  2]
 [ 3  4  5]
 [ 6  7  8]
 [ 9 10 11]]
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[231], line 7
      5 print(a)
      6 print(b)
----> 7 print(a+b)

ValueError: operands could not be broadcast together with shapes (3,4) (4,3)
```

EXPLANATION : there is no 1 to convert ,so got failed

In [232…

```
# doesnt work
a = np.arange(16).reshape(4,4)
b = np.arange(4).reshape(2,2)

print(a)
print(b)
print(a+b)
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]]
[[0 1]
 [2 3]]
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[232], line 7
      5 print(a)
      6 print(b)
----> 7 print(a+b)

ValueError: operands could not be broadcast together with shapes (4,4) (2,2)
```

Working with mathematical formulas

In [233…
```
k = np.arange(10)
```

In [234…
```
k
```

Out[234…
```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

In [235…
```
np.sum(k)
```

Out[235…
```
45
```

In [236…
```
np.sin(k)
```

Out[236…
```
array([ 0.        ,  0.84147098,  0.90929743,  0.14112001, -0.7568025 ,
       -0.95892427, -0.2794155 ,  0.6569866 ,  0.98935825,  0.41211849])
```

In [237…
```
def sigmoid(array):
    return 1/(1+np.exp(-(array)))
k = np.arange(10)
sigmoid(k)
```

Out[237…
```
array([0.5       , 0.73105858, 0.88079708, 0.95257413, 0.98201379,
       0.99330715, 0.99752738, 0.99908895, 0.99966465, 0.99987661])
```

In [238…
```
k=np.arange(100)
sigmoid(k)
```

Out[238…
```
array([0.5       , 0.73105858, 0.88079708, 0.95257413, 0.98201379,
       0.99330715, 0.99752738, 0.99908895, 0.99966465, 0.99987661,
       0.9999546 , 0.9999833 , 0.99999386, 0.99999774, 0.99999917,
       0.99999969, 0.99999989, 0.99999996, 0.99999998, 0.99999999,
       1.        , 1.        , 1.        , 1.        , 1.        ,
       1.        , 1.        , 1.        , 1.        , 1.        ,
       1.        , 1.        , 1.        , 1.        , 1.        ,
       1.        , 1.        , 1.        , 1.        , 1.        ,
       1.        , 1.        , 1.        , 1.        , 1.        ,
       1.        , 1.        , 1.        , 1.        , 1.        ,
       1.        , 1.        , 1.        , 1.        , 1.        ,
       1.        , 1.        , 1.        , 1.        , 1.        ,
       1.        , 1.        , 1.        , 1.        , 1.        ,
       1.        , 1.        , 1.        , 1.        , 1.        ,
       1.        , 1.        , 1.        , 1.        , 1.        ,
       1.        , 1.        , 1.        , 1.        , 1.        ,
       1.        , 1.        , 1.        , 1.        , 1.        ,
       1.        , 1.        , 1.        , 1.        , 1.        ,
       1.        , 1.        , 1.        , 1.        , 1.        ,
       1.        , 1.        , 1.        , 1.        , 1.        ,
       1.        , 1.        , 1.        , 1.        , 1.        ])
```

mean squared error

In [240…
```
actual = np.random.randint(1,50,25)
predicted = np.random.randint(1,50,25)
```

In [241…
```
actual
```

Out[241…
```
array([34, 45,  6, 49, 32, 33,  8,  7, 31, 26, 29, 38, 47,  9, 44, 34, 30,
        1, 44,  1, 12, 22, 10, 34, 44])
```

In [242…
```
predicted
```

Out[242…
```
array([15,  7, 27, 13,  6, 23, 17, 46, 49, 15,  4, 12, 35, 12, 38, 21, 26,
       28, 35,  3, 21, 11, 37, 10, 11])
```

In [245…
```
def mse(actual,predicted):
    return np.mean((actual-predicted)**2)
mse(actual,predicted)
```

Out[245…
```
458.0
```

In [246…
```
actual-predicted
```

Out[246…
```
array([ 19,  38, -21,  36,  26,  10,  -9, -39, -18,  11,  25,  26,  12,
        -3,   6,  13,   4, -27,   9,  -2,  -9,  11, -27,  24,  33])
```

In [247…
```
(actual-predicted)**2
```

Out[247…
```
array([ 361, 1444,  441, 1296,  676,  100,   81, 1521,  324,  121,  625,
        676,  144,    9,   36,  169,   16,  729,   81,    4,   81,  121,
        729,  576, 1089])
```

In [249…
```
np.mean((actual-predicted)**2)
```

Out[249…
```
458.0
```

Working with Missing Values

In [250…
```
# Working with missing values --> np.nan
s=np.array([1,2,3,4,np.nan,6])
s
```

Out[250…
```
array([ 1.,  2.,  3.,  4., nan,  6.])
```

In [251…
```
np.isnan(s)
```

Out[251…
```
array([False, False, False, False,  True, False])
```

In [254…
```
s[np.isnan(s)] # nan values
```

Out[254…
```
array([nan])
```

In [253…
```
s[~np.isnan(s)] # not nan values
```

Out[253…
```
array([1., 2., 3., 4., 6.])
```

Plotting Graphs

In [255…
```
# plotting a 2D plot

x = np.linspace(-10,10,100)
x
```

Out[255…
```
array([-10.        ,  -9.7979798 ,  -9.5959596 ,  -9.39393939,
        -9.19191919,  -8.98989899,  -8.78787879,  -8.58585859,
        -8.38383838,  -8.18181818,  -7.97979798,  -7.77777778,
        -7.57575758,  -7.37373737,  -7.17171717,  -6.96969697,
        -6.76767677,  -6.56565657,  -6.36363636,  -6.16161616,
        -5.95959596,  -5.75757576,  -5.55555556,  -5.35353535,
        -5.15151515,  -4.94949495,  -4.74747475,  -4.54545455,
        -4.34343434,  -4.14141414,  -3.93939394,  -3.73737374,
        -3.53535354,  -3.33333333,  -3.13131313,  -2.92929293,
        -2.72727273,  -2.52525253,  -2.32323232,  -2.12121212,
        -1.91919192,  -1.71717172,  -1.51515152,  -1.31313131,
        -1.11111111,  -0.90909091,  -0.70707071,  -0.50505051,
        -0.3030303 ,  -0.1010101 ,   0.1010101 ,   0.3030303 ,
         0.50505051,   0.70707071,   0.90909091,   1.11111111,
         1.31313131,   1.51515152,   1.71717172,   1.91919192,
         2.12121212,   2.32323232,   2.52525253,   2.72727273,
         2.92929293,   3.13131313,   3.33333333,   3.53535354,
         3.73737374,   3.93939394,   4.14141414,   4.34343434,
         4.54545455,   4.74747475,   4.94949495,   5.15151515,
         5.35353535,   5.55555556,   5.75757576,   5.95959596,
         6.16161616,   6.36363636,   6.56565657,   6.76767677,
         6.96969697,   7.17171717,   7.37373737,   7.57575758,
         7.77777778,   7.97979798,   8.18181818,   8.38383838,
         8.58585859,   8.78787879,   8.98989899,   9.19191919,
         9.39393939,   9.5959596 ,   9.7979798 ,  10.        ])
```

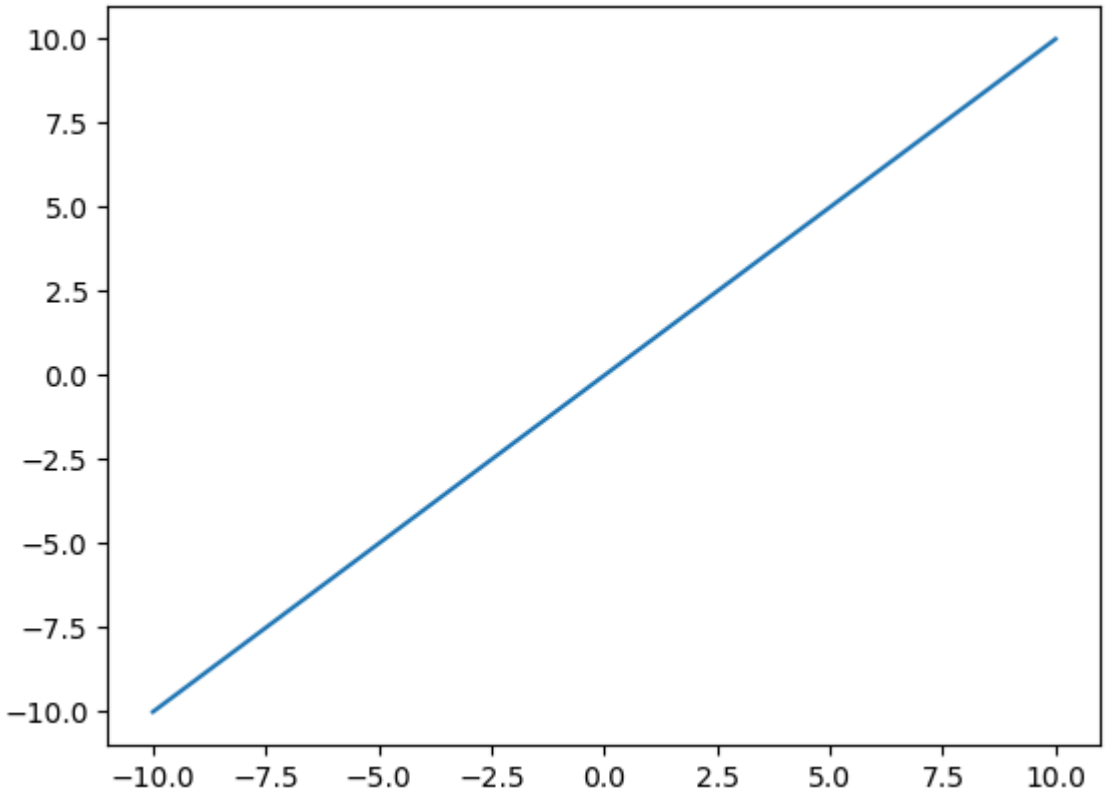
In [256…
```
y = x
```

In [257…
```
y
```

Out[257…
```
array([-10.        ,  -9.7979798 ,  -9.5959596 ,  -9.39393939,
        -9.19191919,  -8.98989899,  -8.78787879,  -8.58585859,
        -8.38383838,  -8.18181818,  -7.97979798,  -7.77777778,
        -7.57575758,  -7.37373737,  -7.17171717,  -6.96969697,
        -6.76767677,  -6.56565657,  -6.36363636,  -6.16161616,
        -5.95959596,  -5.75757576,  -5.55555556,  -5.35353535,
        -5.15151515,  -4.94949495,  -4.74747475,  -4.54545455,
        -4.34343434,  -4.14141414,  -3.93939394,  -3.73737374,
        -3.53535354,  -3.33333333,  -3.13131313,  -2.92929293,
        -2.72727273,  -2.52525253,  -2.32323232,  -2.12121212,
        -1.91919192,  -1.71717172,  -1.51515152,  -1.31313131,
        -1.11111111,  -0.90909091,  -0.70707071,  -0.50505051,
        -0.3030303 ,  -0.1010101 ,   0.1010101 ,   0.3030303 ,
         0.50505051,   0.70707071,   0.90909091,   1.11111111,
         1.31313131,   1.51515152,   1.71717172,   1.91919192,
         2.12121212,   2.32323232,   2.52525253,   2.72727273,
         2.92929293,   3.13131313,   3.33333333,   3.53535354,
         3.73737374,   3.93939394,   4.14141414,   4.34343434,
         4.54545455,   4.74747475,   4.94949495,   5.15151515,
         5.35353535,   5.55555556,   5.75757576,   5.95959596,
         6.16161616,   6.36363636,   6.56565657,   6.76767677,
         6.96969697,   7.17171717,   7.37373737,   7.57575758,
         7.77777778,   7.97979798,   8.18181818,   8.38383838,
         8.58585859,   8.78787879,   8.98989899,   9.19191919,
         9.39393939,   9.5959596 ,   9.7979798 ,  10.        ])
```

In [258…
```
import matplotlib.pyplot as plt
plt.plot(x,y)
```

Out[258…
```
[<matplotlib.lines.Line2D at 0x15181ae6450>]
```

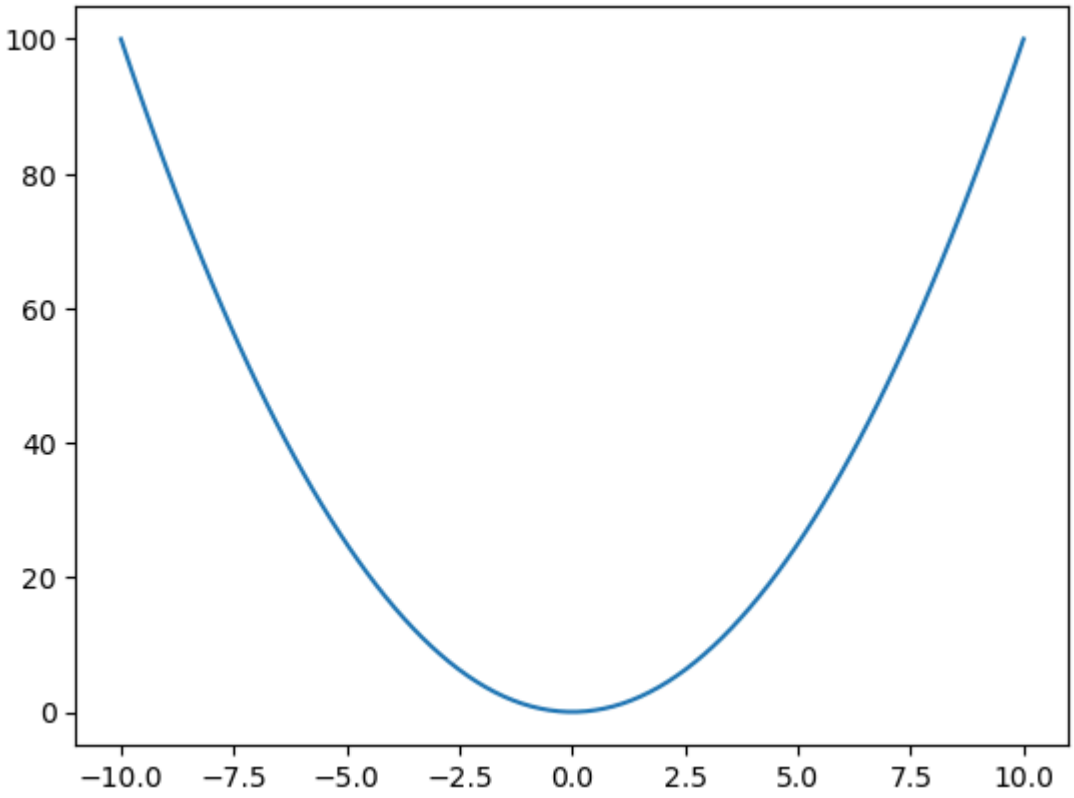


In [260…
```
y =x**2
y
```

Out[260…
```
array([1.00000000e+02, 9.60004081e+01, 9.20824406e+01, 8.82460973e+01,
       8.44913784e+01, 8.08182838e+01, 7.72268136e+01, 7.37169677e+01,
       7.02887460e+01, 6.69421488e+01, 6.36771758e+01, 6.04938272e+01,
       5.73921028e+01, 5.43720029e+01, 5.14335272e+01, 4.85766758e+01,
       4.58014488e+01, 4.31078461e+01, 4.04958678e+01, 3.79655137e+01,
       3.55167840e+01, 3.31496786e+01, 3.08641975e+01, 2.86603408e+01,
       2.65381084e+01, 2.44975003e+01, 2.25385165e+01, 2.06611570e+01,
       1.88654219e+01, 1.71513111e+01, 1.55188246e+01, 1.39679625e+01,
       1.24987246e+01, 1.11111111e+01, 9.80512193e+00, 8.58075707e+00,
       7.43801653e+00, 6.37690032e+00, 5.39740843e+00, 4.49954086e+00,
       3.68329762e+00, 2.94867871e+00, 2.29568411e+00, 1.72431385e+00,
       1.23456790e+00, 8.26446281e-01, 4.99948985e-01, 2.55076013e-01,
       9.18273646e-02, 1.02030405e-02, 1.02030405e-02, 9.18273646e-02,
       2.55076013e-01, 4.99948985e-01, 8.26446281e-01, 1.23456790e+00,
       1.72431385e+00, 2.29568411e+00, 2.94867871e+00, 3.68329762e+00,
       4.49954086e+00, 5.39740843e+00, 6.37690032e+00, 7.43801653e+00,
       8.58075707e+00, 9.80512193e+00, 1.11111111e+01, 1.24987246e+01,
       1.39679625e+01, 1.55188246e+01, 1.71513111e+01, 1.88654219e+01,
       2.06611570e+01, 2.25385165e+01, 2.44975003e+01, 2.65381084e+01,
       2.86603408e+01, 3.08641975e+01, 3.31496786e+01, 3.55167840e+01,
       3.79655137e+01, 4.04958678e+01, 4.31078461e+01, 4.58014488e+01,
       4.85766758e+01, 5.14335272e+01, 5.43720029e+01, 5.73921028e+01,
       6.04938272e+01, 6.36771758e+01, 6.69421488e+01, 7.02887460e+01,
       7.37169677e+01, 7.72268136e+01, 8.08182838e+01, 8.44913784e+01,
       8.82460973e+01, 9.20824406e+01, 9.60004081e+01, 1.00000000e+02])
```

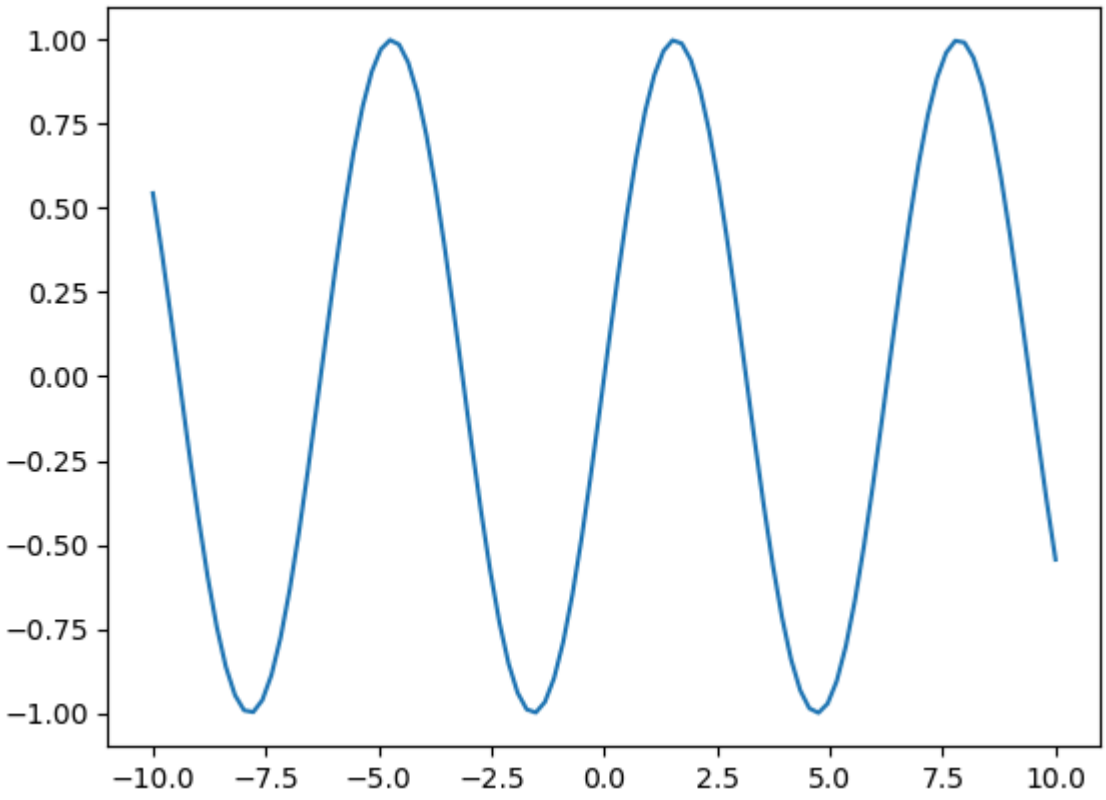
In [259…
```
y = x**2
plt.plot(x,y)
```

Out[259…
```
[<matplotlib.lines.Line2D at 0x151823168a0>]
```



In [262…
```
x
y=np.sin(x)
plt.plot(x,y)
```

Out[262…
```
[<matplotlib.lines.Line2D at 0x15182403ce0>]
```



In [264…
```
y=x*np.log(x)
y
```

```
C:\Users\halvi\AppData\Local\Temp\ipykernel_1456\2577417320.py:1: RuntimeWarning:
invalid value encountered in log
  y=x*np.log(x)
```

Out[264…

```
array([        nan,         nan,         nan,         nan,         nan,
               nan,         nan,         nan,         nan,         nan,
               nan,         nan,         nan,         nan,         nan,
               nan,         nan,         nan,         nan,         nan,
               nan,         nan,         nan,         nan,         nan,
               nan,         nan,         nan,         nan,         nan,
               nan,         nan,         nan,         nan,         nan,
               nan,         nan,         nan,         nan,         nan,
               nan,         nan,         nan,         nan,         nan,
               nan,         nan,         nan,         nan,         nan,
       -0.23156917, -0.36179469, -0.34499841, -0.24508811, -0.08664562,
        0.11706724,  0.35771614,  0.62956885,  0.92843798,  1.25112931,
        1.59512538,  1.95839066,  2.33924512,  2.73627848,  3.14829001,
        3.57424504,  4.01324268,  4.46449148,  4.92729058,  5.40101502,
        5.88510401,  6.37905156,  6.88239878,  7.39472765,  7.91565571,
        8.44483178,  8.98193225,  9.52665793, 10.07873143, 10.6378948 ,
       11.20390763, 11.77654526, 12.35559731, 12.94086634, 13.53216672,
       14.12932359, 14.73217191, 15.34055573, 15.95432739, 16.57334692,
       17.19748143, 17.82660461, 18.46059621, 19.09934166, 19.74273167,
       20.39066184, 21.04303237, 21.69974775, 22.36071651, 23.02585093])
```

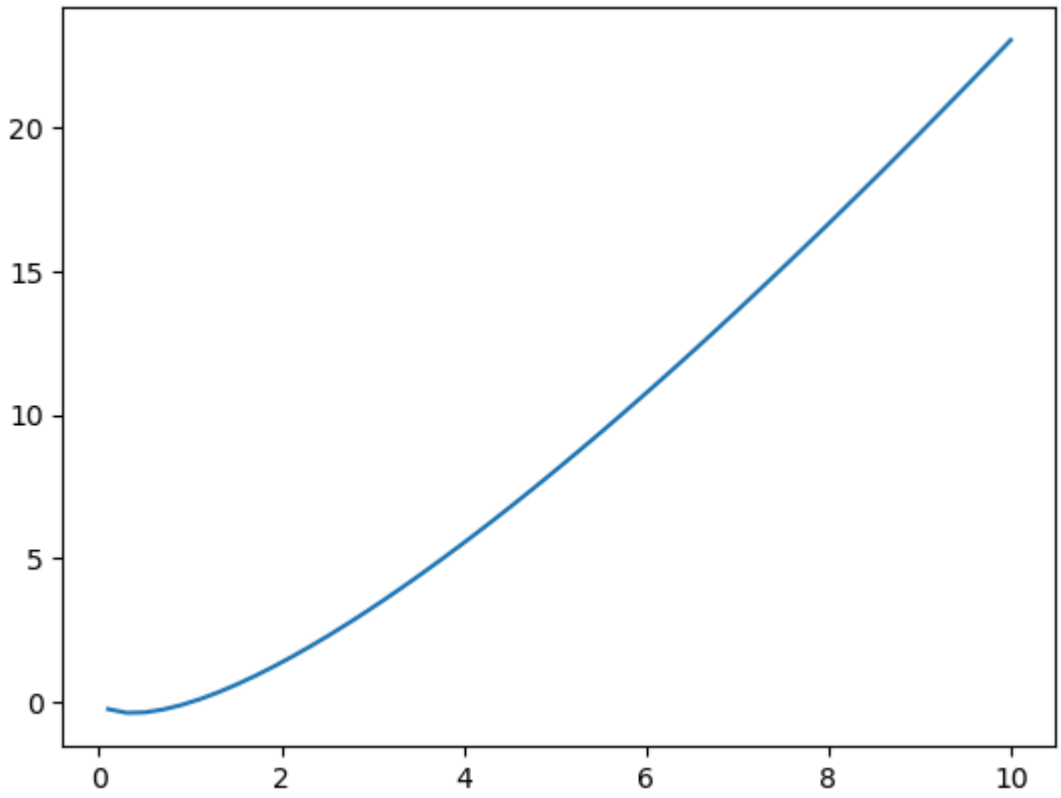
In [265…

```
x=np.linspace(-10,10,100)
y=x*np.log(x)
plt.plot(x,y)
```

```
C:\Users\halvi\AppData\Local\Temp\ipykernel_1456\1310864822.py:2: RuntimeWarning:
invalid value encountered in log
  y=x*np.log(x)
```

Out[265…

```
[<matplotlib.lines.Line2D at 0x15182493110>]
```



In [268…

```
y=1/(1+np.exp(-x))
y
```

Out[268…

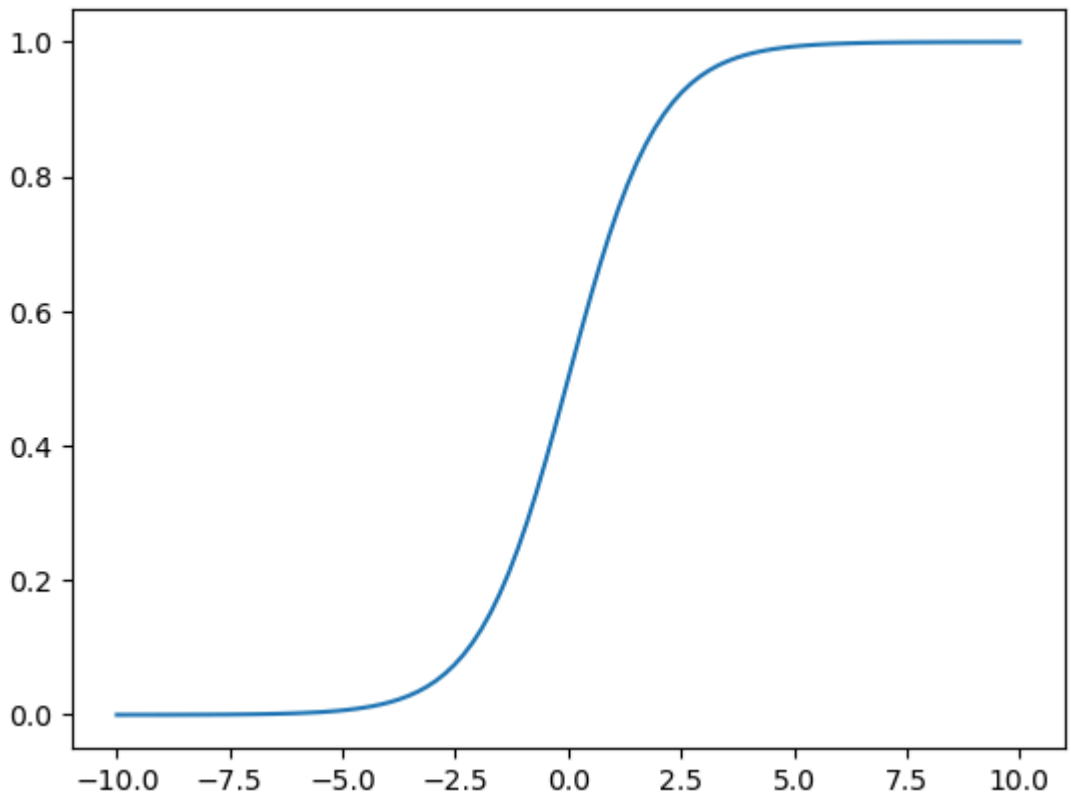
```
array([4.53978687e-05, 5.55606489e-05, 6.79983174e-05, 8.32200197e-05,
       1.01848815e-04, 1.24647146e-04, 1.52547986e-04, 1.86692945e-04,
       2.28478855e-04, 2.79614739e-04, 3.42191434e-04, 4.18766684e-04,
       5.12469082e-04, 6.27124987e-04, 7.67413430e-04, 9.39055039e-04,
       1.14904229e-03, 1.40591988e-03, 1.72012560e-03, 2.10440443e-03,
       2.57431039e-03, 3.14881358e-03, 3.85103236e-03, 4.70911357e-03,
       5.75728612e-03, 7.03711536e-03, 8.59898661e-03, 1.05038445e-02,
       1.28252101e-02, 1.56514861e-02, 1.90885420e-02, 2.32625358e-02,
       2.83228820e-02, 3.44451957e-02, 4.18339400e-02, 5.07243606e-02,
       6.13831074e-02, 7.41067363e-02, 8.92170603e-02, 1.07052146e-01,
       1.27951705e-01, 1.52235823e-01, 1.80176593e-01, 2.11963334e-01,
       2.47663801e-01, 2.87185901e-01, 3.30246430e-01, 3.76354517e-01,
       4.24816868e-01, 4.74768924e-01, 5.25231076e-01, 5.75183132e-01,
       6.23645483e-01, 6.69753570e-01, 7.12814099e-01, 7.52336199e-01,
       7.88036666e-01, 8.19823407e-01, 8.47764177e-01, 8.72048295e-01,
       8.92947854e-01, 9.10782940e-01, 9.25893264e-01, 9.38616893e-01,
       9.49275639e-01, 9.58166060e-01, 9.65554804e-01, 9.71677118e-01,
       9.76737464e-01, 9.80911458e-01, 9.84348514e-01, 9.87174790e-01,
       9.89496155e-01, 9.91401013e-01, 9.92962885e-01, 9.94242714e-01,
       9.95290886e-01, 9.96148968e-01, 9.96851186e-01, 9.97425690e-01,
       9.97895596e-01, 9.98279874e-01, 9.98594080e-01, 9.98850958e-01,
       9.99060945e-01, 9.99232587e-01, 9.99372875e-01, 9.99487531e-01,
       9.99581233e-01, 9.99657809e-01, 9.99720385e-01, 9.99771521e-01,
       9.99813307e-01, 9.99847452e-01, 9.99875353e-01, 9.99898151e-01,
       9.99916780e-01, 9.99932002e-01, 9.99944439e-01, 9.99954602e-01])
```

In [266…

```
# sigmoid
x
y=1/(1+np.exp(-x))
plt.plot(x,y)
```

Out[266…

```
[<matplotlib.lines.Line2D at 0x15181b86870>]
```



In [271…

```
import numpy as np
import matplotlib.pyplot as plt
```

Meshgrid

Meshgrids are a way to create coordinate matrices from coordinate vectors. In NumPy,

the meshgrid function is used to generate a coordinate grid given 1D coordinate arrays. It produces two 2D arrays representing the x and y coordinates of each point on the grid

The np.meshgrid function is used primarily for

Creating/Plotting 2D functions f(x,y) Generating combinations of 2 or more numbers

Example: How you might think to create a 2D function f(x,y)

In [272…

```
x = np.linspace(0,10,100)
y = np.linspace(0,10,100)
```

In [273…

```
x
```

Out[273…

```
array([ 0.        ,  0.1010101 ,  0.2020202 ,  0.3030303 ,  0.4040404 ,
        0.50505051,  0.60606061,  0.70707071,  0.80808081,  0.90909091,
        1.01010101,  1.11111111,  1.21212121,  1.31313131,  1.41414141,
        1.51515152,  1.61616162,  1.71717172,  1.81818182,  1.91919192,
        2.02020202,  2.12121212,  2.22222222,  2.32323232,  2.42424242,
        2.52525253,  2.62626263,  2.72727273,  2.82828283,  2.92929293,
        3.03030303,  3.13131313,  3.23232323,  3.33333333,  3.43434343,
        3.53535354,  3.63636364,  3.73737374,  3.83838384,  3.93939394,
        4.04040404,  4.14141414,  4.24242424,  4.34343434,  4.44444444,
        4.54545455,  4.64646465,  4.74747475,  4.84848485,  4.94949495,
        5.05050505,  5.15151515,  5.25252525,  5.35353535,  5.45454545,
        5.55555556,  5.65656566,  5.75757576,  5.85858586,  5.95959596,
        6.06060606,  6.16161616,  6.26262626,  6.36363636,  6.46464646,
        6.56565657,  6.66666667,  6.76767677,  6.86868687,  6.96969697,
        7.07070707,  7.17171717,  7.27272727,  7.37373737,  7.47474747,
        7.57575758,  7.67676768,  7.77777778,  7.87878788,  7.97979798,
        8.08080808,  8.18181818,  8.28282828,  8.38383838,  8.48484848,
        8.58585859,  8.68686869,  8.78787879,  8.88888889,  8.98989899,
        9.09090909,  9.19191919,  9.29292929,  9.39393939,  9.49494949,
        9.5959596 ,  9.6969697 ,  9.7979798 ,  9.8989899 , 10.        ])
```

In [274…

```
y
```

Out[274…

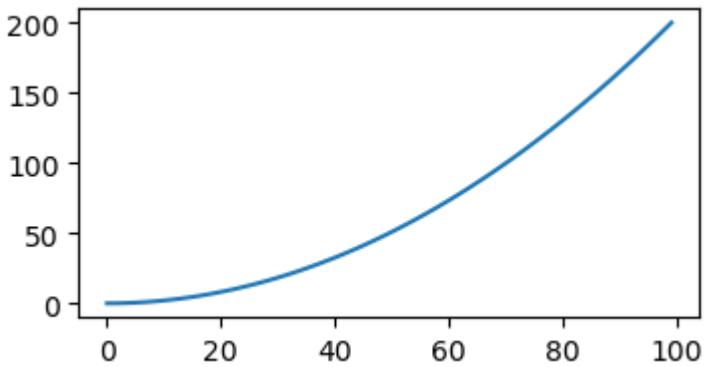
```
array([ 0.        ,  0.1010101 ,  0.2020202 ,  0.3030303 ,  0.4040404 ,
        0.50505051,  0.60606061,  0.70707071,  0.80808081,  0.90909091,
        1.01010101,  1.11111111,  1.21212121,  1.31313131,  1.41414141,
        1.51515152,  1.61616162,  1.71717172,  1.81818182,  1.91919192,
        2.02020202,  2.12121212,  2.22222222,  2.32323232,  2.42424242,
        2.52525253,  2.62626263,  2.72727273,  2.82828283,  2.92929293,
        3.03030303,  3.13131313,  3.23232323,  3.33333333,  3.43434343,
        3.53535354,  3.63636364,  3.73737374,  3.83838384,  3.93939394,
        4.04040404,  4.14141414,  4.24242424,  4.34343434,  4.44444444,
        4.54545455,  4.64646465,  4.74747475,  4.84848485,  4.94949495,
        5.05050505,  5.15151515,  5.25252525,  5.35353535,  5.45454545,
        5.55555556,  5.65656566,  5.75757576,  5.85858586,  5.95959596,
        6.06060606,  6.16161616,  6.26262626,  6.36363636,  6.46464646,
        6.56565657,  6.66666667,  6.76767677,  6.86868687,  6.96969697,
        7.07070707,  7.17171717,  7.27272727,  7.37373737,  7.47474747,
        7.57575758,  7.67676768,  7.77777778,  7.87878788,  7.97979798,
        8.08080808,  8.18181818,  8.28282828,  8.38383838,  8.48484848,
        8.58585859,  8.68686869,  8.78787879,  8.88888889,  8.98989899,
        9.09090909,  9.19191919,  9.29292929,  9.39393939,  9.49494949,
        9.5959596 ,  9.6969697 ,  9.7979798 ,  9.8989899 , 10.        ])
```

In [276…

```
# Try to create 2D function
f = x**2+y**2
f
```

Out[276…
```
array([0.00000000e+00, 2.04060810e-02, 8.16243240e-02, 1.83654729e-01,
       3.26497296e-01, 5.10152025e-01, 7.34618916e-01, 9.99897970e-01,
       1.30598918e+00, 1.65289256e+00, 2.04060810e+00, 2.46913580e+00,
       2.93847567e+00, 3.44862769e+00, 3.99959188e+00, 4.59136823e+00,
       5.22395674e+00, 5.89735741e+00, 6.61157025e+00, 7.36659525e+00,
       8.16243240e+00, 8.99908173e+00, 9.87654321e+00, 1.07948169e+01,
       1.17539027e+01, 1.27538006e+01, 1.37945108e+01, 1.48760331e+01,
       1.59983675e+01, 1.71615141e+01, 1.83654729e+01, 1.96102439e+01,
       2.08958270e+01, 2.22222222e+01, 2.35894297e+01, 2.49974492e+01,
       2.64462810e+01, 2.79359249e+01, 2.94663810e+01, 3.10376492e+01,
       3.26497296e+01, 3.43026222e+01, 3.59963269e+01, 3.77308438e+01,
       3.95061728e+01, 4.13223140e+01, 4.31792674e+01, 4.50770330e+01,
       4.70156107e+01, 4.89950005e+01, 5.10152025e+01, 5.30762167e+01,
       5.51780431e+01, 5.73206816e+01, 5.95041322e+01, 6.17283951e+01,
       6.39934701e+01, 6.62993572e+01, 6.86460565e+01, 7.10335680e+01,
       7.34618916e+01, 7.59310274e+01, 7.84409754e+01, 8.09917355e+01,
       8.35833078e+01, 8.62156923e+01, 8.88888889e+01, 9.16028977e+01,
       9.43577186e+01, 9.71533517e+01, 9.99897970e+01, 1.02867054e+02,
       1.05785124e+02, 1.08744006e+02, 1.11743700e+02, 1.14784206e+02,
       1.17865524e+02, 1.20987654e+02, 1.24150597e+02, 1.27354352e+02,
       1.30598918e+02, 1.33884298e+02, 1.37210489e+02, 1.40577492e+02,
       1.43985308e+02, 1.47433935e+02, 1.50923375e+02, 1.54453627e+02,
       1.58024691e+02, 1.61636568e+02, 1.65289256e+02, 1.68982757e+02,
       1.72717070e+02, 1.76492195e+02, 1.80308132e+02, 1.84164881e+02,
       1.88062443e+02, 1.92000816e+02, 1.95980002e+02, 2.00000000e+02])
```

In [279…
```
# plot
plt.figure(figsize=(4,2))
plt.plot(f)
plt.show()
```



but f is a 1 dimensional function! How does one generate a surface plot?

In [281…
```
x = np.arange(3)
y = np.arange(3)
```

In [282…
```
x
```

Out[282…
```
array([0, 1, 2])
```

In [283…
```
y
```

Out[283…
```
array([0, 1, 2])
```

Generating a meshgrid:

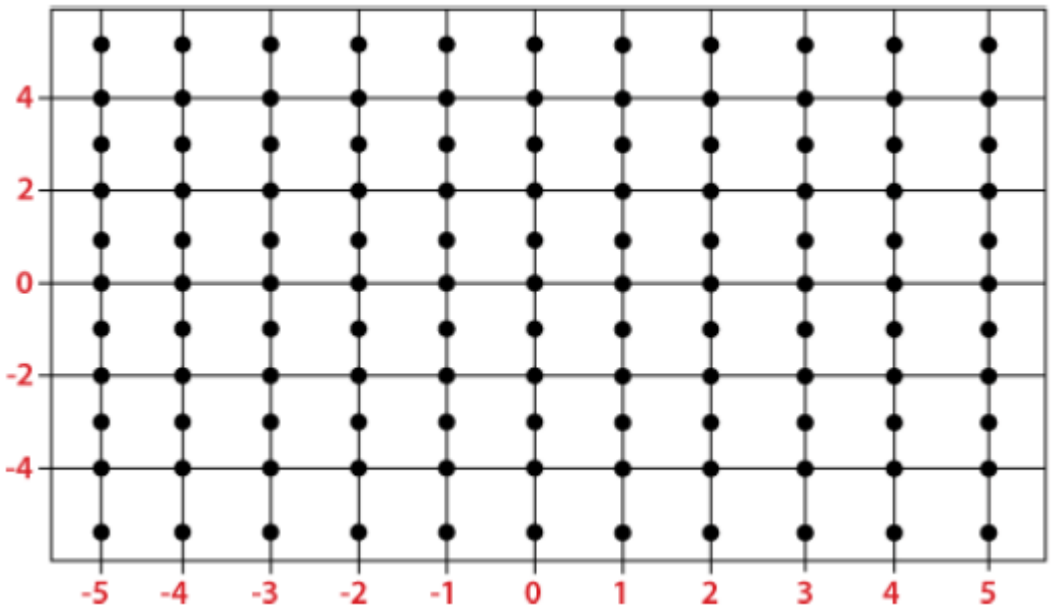
In [284…
```
xv ,yv= np.meshgrid(x,y)
```

In [285…
```
xv
```

Out[285…
```
array([[0, 1, 2],
       [0, 1, 2],
       [0, 1, 2]])
```

In [286…
```
yv
```

Out[286…
```
array([[0, 0, 0],
       [1, 1, 1],
       [2, 2, 2]])
```



In [287…
```
P = np.linspace(-4, 4, 9)
V = np.linspace(-5, 5, 11)
print(P)
print(V)
```

```
[-4. -3. -2. -1.  0.  1.  2.  3.  4.]
[-5. -4. -3. -2. -1.  0.  1.  2.  3.  4.  5.]
```

In [288…
```
P_1,V_1=np.meshgrid(P,V)
```

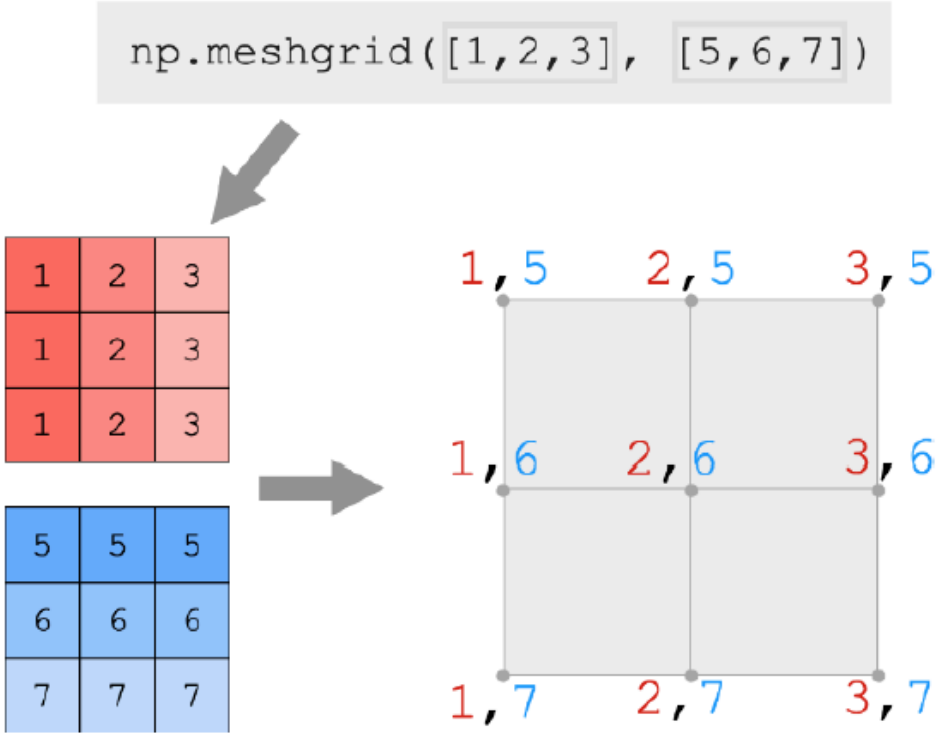
In [292…
```
print(P_1)
```

```
[[-4. -3. -2. -1.  0.  1.  2.  3.  4.]
 [-4. -3. -2. -1.  0.  1.  2.  3.  4.]
 [-4. -3. -2. -1.  0.  1.  2.  3.  4.]
 [-4. -3. -2. -1.  0.  1.  2.  3.  4.]
 [-4. -3. -2. -1.  0.  1.  2.  3.  4.]
 [-4. -3. -2. -1.  0.  1.  2.  3.  4.]
 [-4. -3. -2. -1.  0.  1.  2.  3.  4.]
 [-4. -3. -2. -1.  0.  1.  2.  3.  4.]
 [-4. -3. -2. -1.  0.  1.  2.  3.  4.]
 [-4. -3. -2. -1.  0.  1.  2.  3.  4.]
 [-4. -3. -2. -1.  0.  1.  2.  3.  4.]]
```

In [294…
```
V_1
```

Out[294…
```
array([[-5., -5., -5., -5., -5., -5., -5., -5., -5.],
       [-4., -4., -4., -4., -4., -4., -4., -4., -4.],
       [-3., -3., -3., -3., -3., -3., -3., -3., -3.],
       [-2., -2., -2., -2., -2., -2., -2., -2., -2.],
       [-1., -1., -1., -1., -1., -1., -1., -1., -1.],
       [ 0.,  0.,  0.,  0.,  0.,  0.,  0.,  0.,  0.],
       [ 1.,  1.,  1.,  1.,  1.,  1.,  1.,  1.,  1.],
       [ 2.,  2.,  2.,  2.,  2.,  2.,  2.,  2.,  2.],
       [ 3.,  3.,  3.,  3.,  3.,  3.,  3.,  3.,  3.],
       [ 4.,  4.,  4.,  4.,  4.,  4.,  4.,  4.,  4.],
       [ 5.,  5.,  5.,  5.,  5.,  5.,  5.,  5.,  5.]])
```

Numpy Meshgrid Creates Coordinates for a Grid System



These arrays, xv and yv, each seperately give the x and y coordinates on a 2D grid. You can do normal numpy operations on these arrays:

In [295…
```
xv**2+yv**2
```

Out[295…
```
array([[0, 1, 4],
       [1, 2, 5],
       [4, 5, 8]])
```

This can be done on a larger scale to plot surface plots of 2D functions Generate functions $f(x, y) = e-(x^2+y^2)$ for $-2 \le x \le 2$ and $-1 \le y \le 1$

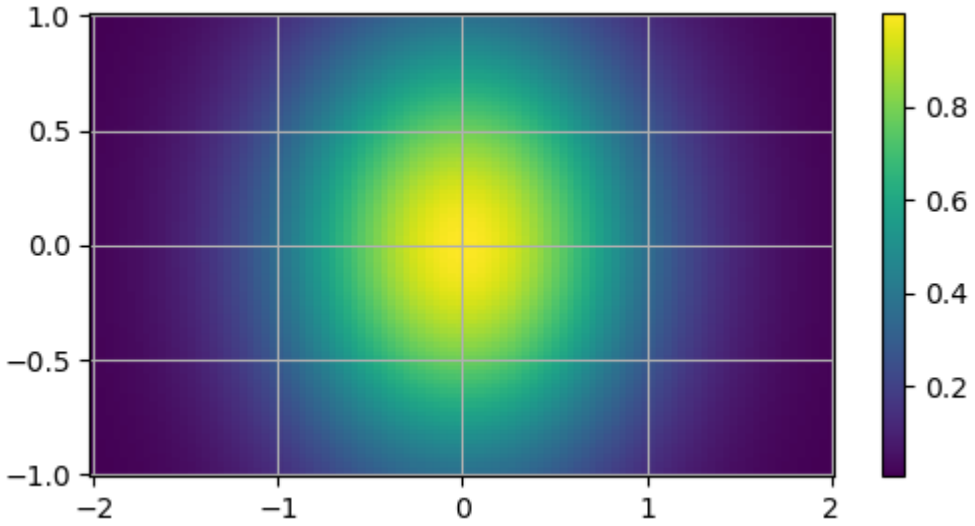
In [296…
```
x= np.linspace(-2,2,100)
y = np.linspace(-1,1,100)
xv, yv = np.meshgrid(x, y)
f = np.exp( -xv**2-yv**2)
```

Note: pcolormesh is typically the preferable function for 2D plotting, as opposed to imshow or pcolor, which take longer.)

In [298…

```
plt.figure(figsize=(6 , 3))
plt.pcolormesh(xv,yv, f, shading=' auto')
plt.colorbar()
plt.grid()
plt.show()
```

```
C:\Users\halvi\AppData\Local\Temp\ipykernel_1456\916373200.py:2: UserWarning: sha
ding value ' auto' not in list of valid values ['gouraud', 'nearest', 'flat', 'au
to']. Setting shading='auto'.
  plt.pcolormesh(xv,yv, f, shading=' auto')
```



f(x,y) = 1 & x^2+y^2 < 1 \ 0 & x^2+y^2

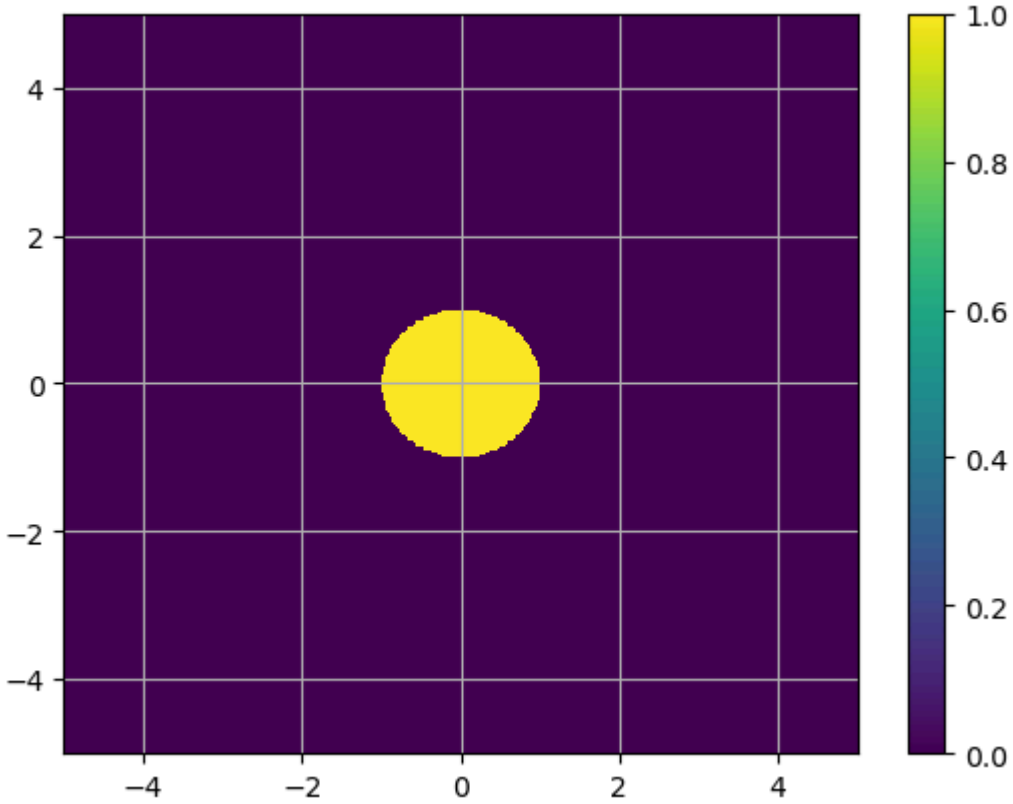
In [3]:

```
import numpy as np
import matplotlib.pyplot as plt
def f (x, y):
    return np.where((x**2 + y**2 < 1 ), 1.0 , 0.0)

x = np.linspace(-5 , 5 , 500)
y = np.linspace(-5 , 5 , 500 )
xv, yv = np.meshgrid(x, y)
rectangular_mask=f(xv, yv)

plt.pcolormesh(xv,yv, rectangular_mask, shading =' auto')
plt.colorbar()
plt.grid()
plt.show()
```

```
C:\Users\halvi\AppData\Local\Temp\ipykernel_10664\2364529418.py:11: UserWarning:
shading value ' auto' not in list of valid values ['gouraud', 'nearest', 'flat',
'auto']. Setting shading='auto'.
  plt.pcolormesh(xv,yv, rectangular_mask, shading =' auto')
```

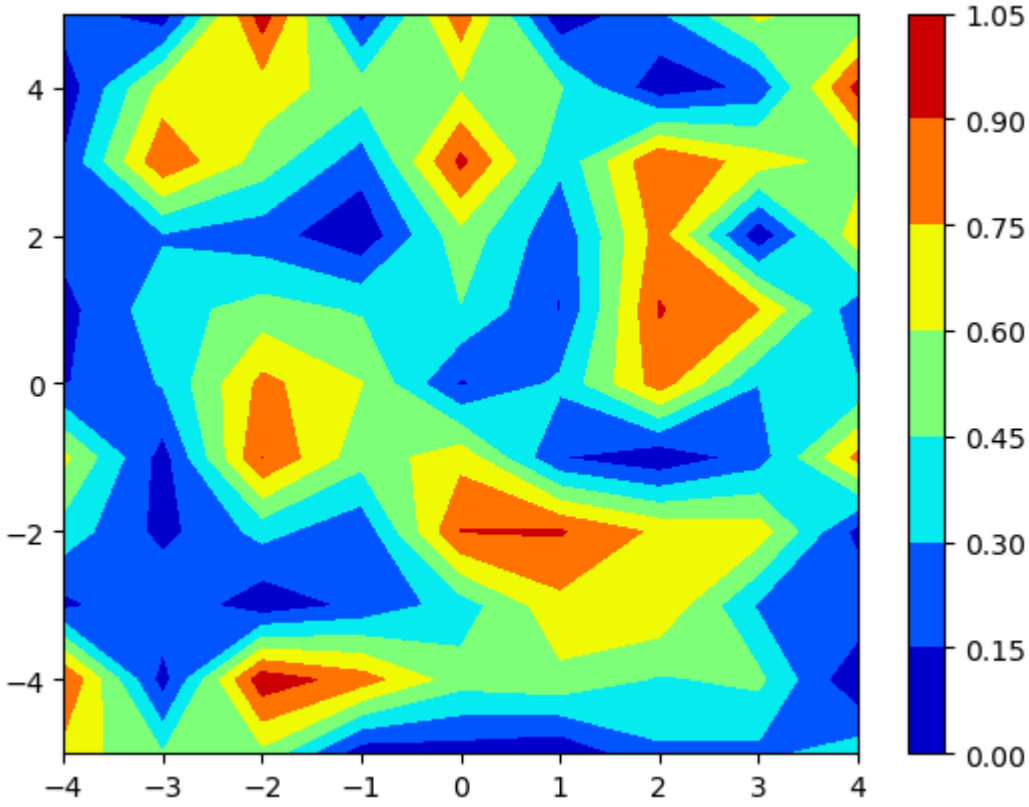


In [4]:
```
#numpy.linspace creates an array of
# 9 linearly placed elements between
# 4 and 4, both inclusive
x = np.linspace(-4 , 4 , 9)
```

In [5]:
```
y = np.linspace(-5 , 5 , 11)
```

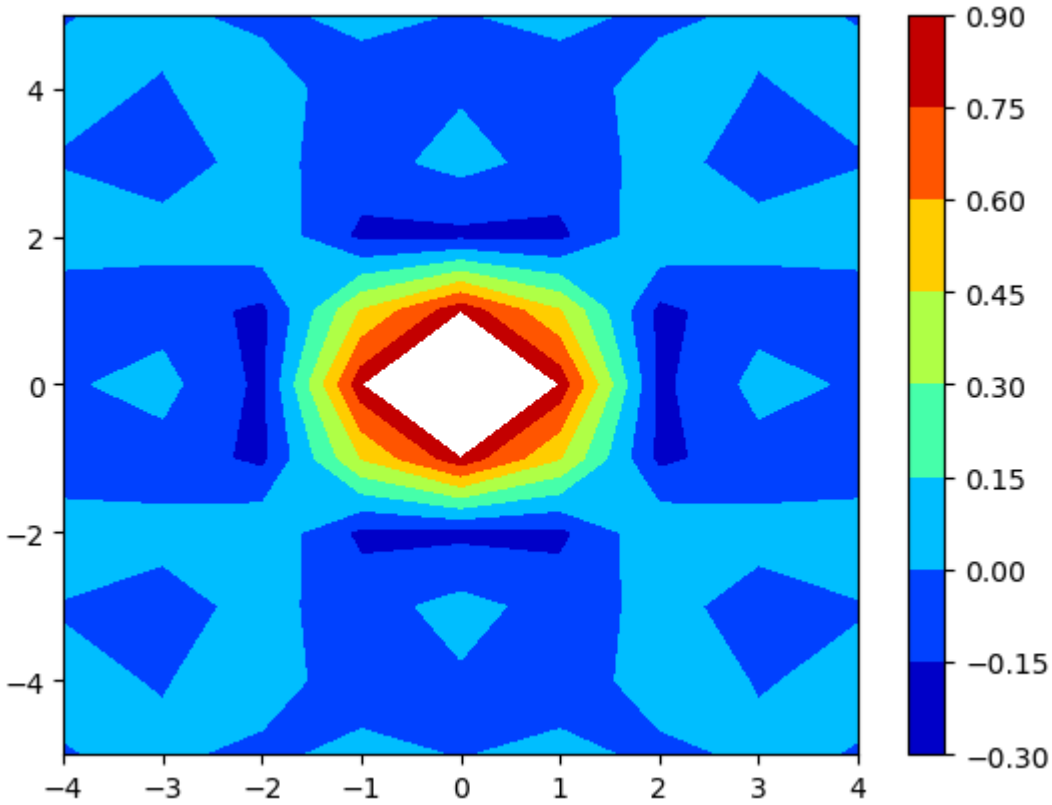
In [6]:
```
x_1, y_1 = np.meshgrid(x, y)
```

In [7]:
```
random_data= np.random.random(( 11 , 9))
plt.contourf(x_1, y_1, random_data, cmap = 'jet')
plt.colorbar()
plt.show()
```



In [10]:
```
sine= (np.sin(x_1**2+y_1**2 ))/(x_1**2+y_1**2 )
plt.contourf(x_1, y_1, sine, cmap = 'jet')
plt.colorbar()
plt.show()
```

```
C:\Users\halvi\AppData\Local\Temp\ipykernel_10664\2583048474.py:1: RuntimeWarnin
g: invalid value encountered in divide
  sine= (np.sin(x_1**2+y_1**2 ))/(x_1**2+y_1**2 )
```



We observe that x_1 is a row repeated matrix whereas y_1 is a column repeated matrix. One row of x_1 and one column of y_1 is enough to determine the positions of all the points as the other values will get repeated over and over.

In [11]:
```
x_1, y_1= np.meshgrid(x, y, sparse = True)
```

In [12]:
```
x_1
```

Out[12]:
```
array([[-4., -3., -2., -1.,  0.,  1.,  2.,  3.,  4.]])
```

In [13]:
```
y_1
```

Out[13]:
```
array([[-5.],
       [-4.],
       [-3.],
       [-2.],
       [-1.],
       [ 0.],
       [ 1.],
       [ 2.],
       [ 3.],
       [ 4.],
       [ 5.]])
```

The shape of x_1 changed from (11, 9) to (1, 9) and that of y_1 changed from (11, 9) to (11, 1) The indexing of Matrix is however different. Actually, it is the exact opposite of Cartesian indexing.

np.sort

Return a sorted copy of an array.

In [15]:
```
a = np.random.randint(1,100,15) #1D
a
```

Out[15]:
```
array([25, 73, 65, 62, 10, 59, 79, 18,  3, 66, 81, 70, 40, 85, 38])
```

In [16]:
```
b = np.random.randint(1,100,24).reshape(6,4) #2d
b
```

Out[16]:
```
array([[26, 94, 41, 26],
       [57, 10, 62, 11],
       [44, 50, 39, 17],
       [82, 40, 51, 27],
       [38, 51, 42, 45],
       [69, 94, 51, 87]])
```

In [17]:
```
np.sort(a) # Default= Ascending
```

Out[17]:
```
array([ 3, 10, 18, 25, 38, 40, 59, 62, 65, 66, 70, 73, 79, 81, 85])
```

In [18]:
```
np.sort(a)[::-1] # Descending order
```

Out[18]:
```
array([85, 81, 79, 73, 70, 66, 65, 62, 59, 40, 38, 25, 18, 10,  3])
```

In [19]:
```
np.sort(b) # row rise sorting
```

Out[19]:
```
array([[26, 26, 41, 94],
       [10, 11, 57, 62],
       [17, 39, 44, 50],
       [27, 40, 51, 82],
       [38, 42, 45, 51],
       [51, 69, 87, 94]])
```

In [20]:
```
np.sort(b,axis = 0) # column rise sorting
```

Out[20]:
```
array([[26, 10, 39, 11],
       [38, 40, 41, 17],
       [44, 50, 42, 26],
       [57, 51, 51, 27],
       [69, 94, 51, 45],
       [82, 94, 62, 87]])
```

np.append

The numpy.append() appends values along the mentioned axis at the end of the array

In [21]:
```
a
```

Out[21]:
```
array([25, 73, 65, 62, 10, 59, 79, 18,  3, 66, 81, 70, 40, 85, 38])
```

In [22]:
```
np.append(a,200)
```

Out[22]:
```
array([ 25,  73,  65,  62,  10,  59,  79,  18,   3,  66,  81,  70,  40,
        85,  38, 200])
```

In [23]:
```
b
```

Out[23]:
```
array([[26, 94, 41, 26],
       [57, 10, 62, 11],
       [44, 50, 39, 17],
       [82, 40, 51, 27],
       [38, 51, 42, 45],
       [69, 94, 51, 87]])
```

In [24]:
```
# Adding Extra column :1
np.append(b,np.ones((b.shape[0],1)))
```

Out[24]:
```
array([26., 94., 41., 26., 57., 10., 62., 11., 44., 50., 39., 17., 82.,
       40., 51., 27., 38., 51., 42., 45., 69., 94., 51., 87.,  1.,  1.,
        1.,  1.,  1.,  1.])
```

In [28]:
```
np.append(b,np.ones((b.shape[0],1)),axis=1)
```

Out[28]:
```
array([[26., 94., 41., 26.,  1.],
       [57., 10., 62., 11.,  1.],
       [44., 50., 39., 17.,  1.],
       [82., 40., 51., 27.,  1.],
       [38., 51., 42., 45.,  1.],
       [69., 94., 51., 87.,  1.]])
```

In [30]:
```
#Adding random numbers in new column
np.append(b,np.random.random((b.shape[0],1)),axis=1)
```

Out[30]:
```
array([[26.        , 94.        , 41.        , 26.        ,  0.61038413],
       [57.        , 10.        , 62.        , 11.        ,  0.27599826],
       [44.        , 50.        , 39.        , 17.        ,  0.78461972],
       [82.        , 40.        , 51.        , 27.        ,  0.79771613],
       [38.        , 51.        , 42.        , 45.        ,  0.79864704],
       [69.        , 94.        , 51.        , 87.        ,  0.38355605]])
```

np.concatenate

numpy.concatenate() function concatenate a sequence of arrays along an existing axis.

In [32]:
```
#code
c = np.arange( 6 ).reshape(2,3)
d= np.arange( 6,12 ).reshape(2,3)
```

In [33]:
```
c
```

Out[33]:
```
array([[0, 1, 2],
       [3, 4, 5]])
```

In [34]:
```
d
```

Out[34]:
```
array([[ 6,  7,  8],
       [ 9, 10, 11]])
```

we can use it replacement of vstack and hstack

In [35]:
```
np.concatenate((c,d))# Row wise
```

Out[35]:
```
array([[ 0,  1,  2],
       [ 3,  4,  5],
       [ 6,  7,  8],
       [ 9, 10, 11]])
```

In [36]:
```
np.concatenate((c,d),axis=1 ) # column wise
```

Out[36]:
```
array([[ 0,  1,  2,  6,  7,  8],
       [ 3,  4,  5,  9, 10, 11]])
```

np.unique

With the help of np.unique() method, we can get the unique values from an array given as parameter in np.unique() method.

In [37]:
```
# code
e = np.array([1,1,2,2,3,3,4,4,5,5,6,6])
```

In [38]:
```
e
```

Out[38]:
```
array([1, 1, 2, 2, 3, 3, 4, 4, 5, 5, 6, 6])
```

In [39]:
```
np.unique(e)
```

Out[39]:
```
array([1, 2, 3, 4, 5, 6])
```

np.expand_dims

With the help of Numpy.expand_dims() method, we can get the expanded dimensions of an array

In [40]:
```
a
```

Out[40]:
```
array([25, 73, 65, 62, 10, 59, 79, 18,  3, 66, 81, 70, 40, 85, 38])
```

In [41]:
```
a.shape
```

Out[41]:
```
(15,)
```

In [44]:
```
# converting into 2D array
np.expand_dims(a,axis= 0)
```

Out[44]:
```
array([[25, 73, 65, 62, 10, 59, 79, 18,  3, 66, 81, 70, 40, 85, 38]])
```

In [45]:
```
np.expand_dims(a,axis= 0 ).shape # 2D
```

Out[45]:
```
(1, 15)
```

In [46]:
```
np.expand_dims(a,axis= 1)
```

Out[46]:
```
array([[25],
       [73],
       [65],
       [62],
       [10],
       [59],
       [79],
       [18],
       [ 3],
       [66],
       [81],
       [70],
       [40],
       [85],
       [38]])
```

In [47]:
```
np.expand_dims(a,axis= 1).shape
```

Out[47]:
```
(15, 1)
```

np.where

The numpy.where() function returns the indices of elements in an input array where the given condition is satisfied.

In [48]:
```
a
```

Out[48]:
```
array([25, 73, 65, 62, 10, 59, 79, 18,  3, 66, 81, 70, 40, 85, 38])
```

In [50]:
```
np.where(a>50) # find all indices with value greater than
50
```

Out[50]:
```
50
```

syntax np.where(condition, True , false)

In [51]:
```
# replace all values > 50 with 0
np.where(a>50,0,a)
```

Out[51]:
```
array([25,  0,  0,  0, 10,  0,  0, 18,  3,  0,  0,  0, 40,  0, 38])
```

In [52]:
```
# print and replace all even numbers to 0
np.where(a%2 == 0,0,a)
```

Out[52]:
```
array([25, 73, 65,  0,  0, 59, 79,  0,  3,  0, 81,  0,  0, 85,  0])
```

np.argmax

The numpy.argmax() function returns indices of the max element of the array in a particular axis.

arg = argument

In [53]:
```
a
```

Out[53]:
```
array([25, 73, 65, 62, 10, 59, 79, 18,  3, 66, 81, 70, 40, 85, 38])
```

In [54]:
```
np.argmax(a)
# biggest number : index number
```

Out[54]:
```
13
```

In [55]:
```
b
```

Out[55]:
```
array([[26, 94, 41, 26],
       [57, 10, 62, 11],
       [44, 50, 39, 17],
       [82, 40, 51, 27],
       [38, 51, 42, 45],
       [69, 94, 51, 87]])
```

In [56]:
```
np.argmax(b,axis=1 ) # row wise bigest number : index
```

Out[56]:
```
array([1, 2, 1, 0, 1, 1], dtype=int64)
```

In [57]:
```
np.argmax(b,axis=0 ) # column wise bigest number : index
```

Out[57]:
```
array([3, 0, 1, 5], dtype=int64)
```

In [58]:
```
np.argmin(a)
```

Out[58]:
```
8
```

On Statistics:

np.cumsum

numpy.cumsum() function is used when we want to compute the cumulative sum of array elements over a given axis.

In [59]:
```
a
```

Out[59]:
```
array([25, 73, 65, 62, 10, 59, 79, 18,  3, 66, 81, 70, 40, 85, 38])
```

In [61]:
```
np.cumsum(a)
```

Out[61]:
```
array([ 25,  98, 163, 225, 235, 294, 373, 391, 394, 460, 541, 611, 651,
       736, 774])
```

In [62]:
```
b
```

Out[62]:
```
array([[26, 94, 41, 26],
       [57, 10, 62, 11],
       [44, 50, 39, 17],
       [82, 40, 51, 27],
       [38, 51, 42, 45],
       [69, 94, 51, 87]])
```

In [63]:
```
np.cumsum(b)
```

Out[63]:
```
array([  26,  120,  161,  187,  244,  254,  316,  327,  371,  421,  460,
        477,  559,  599,  650,  677,  715,  766,  808,  853,  922, 1016,
       1067, 1154])
```

In [64]:
```
np.cumsum(b,axis=1 ) # row wise calculation or cumulative sum
```

Out[64]:
```
array([[ 26, 120, 161, 187],
       [ 57,  67, 129, 140],
       [ 44,  94, 133, 150],
       [ 82, 122, 173, 200],
       [ 38,  89, 131, 176],
       [ 69, 163, 214, 301]])
```

In [65]:
```
np.cumsum(b,axis=0 ) # column wise calculation or cumulative sum
```

Out[65]:
```
array([[ 26,  94,  41,  26],
       [ 83, 104, 103,  37],
       [127, 154, 142,  54],
       [209, 194, 193,  81],
       [247, 245, 235, 126],
       [316, 339, 286, 213]])
```

In [66]:
```
# np.cumprod ----> Multiply
np.cumprod(a)
```

Out[66]:
```
array([         25,        1825,      118625,     7354750,    73547500,
        44335204,  -792486180, -1379849352,   155419240,  1667735248,
      1942568912, -1459129632,  1764356864,  -353521920,  -548931072])
```

np.percentile

numpy.percentile()function used to compute the nth percentile of the given data (array elements) along the specified axis.

In [67]:
```
a
```

Out[67]:
```
array([25, 73, 65, 62, 10, 59, 79, 18,  3, 66, 81, 70, 40, 85, 38])
```

In [68]:
```
np.percentile(a,100 ) # Max
```

Out[68]:
```
85.0
```

In [69]:
```
np.percentile(a,0 ) # Min
```

Out[69]:
```
3.0
```

In [70]:
```
np.percentile(a,50 ) # Median
```

Out[70]:
```
62.0
```

In [71]:
```
np.median(a)
```

Out[71]:
```
62.0
```

np.histogram

Numpy has a built-in numpy.histogram() function which represents the frequency of data distribution in the graphical form.

In [72]:
```
a
```

Out[72]:
```
array([25, 73, 65, 62, 10, 59, 79, 18,  3, 66, 81, 70, 40, 85, 38])
```

In [74]:
```
np.histogram(a , bins= [10,20,30,40,50,60,70,80,90,100])
```

Out[74]:
```
(array([2, 1, 1, 1, 1, 3, 3, 2, 0], dtype=int64),
 array([ 10,  20,  30,  40,  50,  60,  70,  80,  90, 100]))
```

In [75]:
```
np.histogram(a , bins= [0,50,100])
```

Out[75]:
```
(array([6, 9], dtype=int64), array([  0,  50, 100]))
```

np.corrcoef

Return Pearson product-moment correlation coefficients.

In [76]:
```
salary= np.array([ 20000,40000,25000,35000,60000])
experience= np.array([ 1,3,2,4,2])
```

In [77]:
```
salary
```

Out[77]:
```
array([20000, 40000, 25000, 35000, 60000])
```

In [78]:
```
np.corrcoef(salary,experience)# Correlation Coefficient
```

Out[78]:
```
array([[1.        , 0.25344572],
       [0.25344572, 1.        ]])
```

Utility functions

np.isin

With the help of numpy.isin() method, we can see that one array having values are checked in a different numpy array having different elements with different sizes.

In [79]:
```
a
```

Out[79]:
```
array([25, 73, 65, 62, 10, 59, 79, 18,  3, 66, 81, 70, 40, 85, 38])
```

In [80]:
```
items= [10,20,30,40,50,60,70,80,90,100]
np.isin(a,items)
```

Out[80]:
```
array([False, False, False, False,  True, False, False, False, False,
       False, False,  True,  True, False, False])
```

In [81]:
```
a[np.isin(a,items)]
```

Out[81]:
```
array([10, 70, 40])
```

np.flip

The numpy.flip() function reverses the order of array elements along the specified axis, preserving the shape of the array.

In [82]:
```
a
```

Out[82]:
```
array([25, 73, 65, 62, 10, 59, 79, 18,  3, 66, 81, 70, 40, 85, 38])
```

In [83]:
```
np.flip(a) # reverse
```

Out[83]:
```
array([38, 85, 40, 70, 81, 66,  3, 18, 79, 59, 10, 62, 65, 73, 25])
```

In [84]:
```
b
```

Out[84]:
```
array([[26, 94, 41, 26],
       [57, 10, 62, 11],
       [44, 50, 39, 17],
       [82, 40, 51, 27],
       [38, 51, 42, 45],
       [69, 94, 51, 87]])
```

In [85]:
```
np.flip(b)
```

Out[85]:
```
array([[87, 51, 94, 69],
       [45, 42, 51, 38],
       [27, 51, 40, 82],
       [17, 39, 50, 44],
       [11, 62, 10, 57],
       [26, 41, 94, 26]])
```

In [86]:
```
np.flip(b,axis = 1) # row
```

Out[86]:
```
array([[26, 41, 94, 26],
       [11, 62, 10, 57],
       [17, 39, 50, 44],
       [27, 51, 40, 82],
       [45, 42, 51, 38],
       [87, 51, 94, 69]])
```

In [87]:
```
np.flip(b,axis = 0 ) # column
```

Out[87]:
```
array([[69, 94, 51, 87],
       [38, 51, 42, 45],
       [82, 40, 51, 27],
       [44, 50, 39, 17],
       [57, 10, 62, 11],
       [26, 94, 41, 26]])
```

np.put

The numpy.put() function replaces specific elements of an array with given values of p_array. Array indexed works on flattened array.

In [88]:
```
a
```

Out[88]:
```
array([25, 73, 65, 62, 10, 59, 79, 18,  3, 66, 81, 70, 40, 85, 38])
```

In [89]:
```
np.put(a,[0,1],[110,530]) # permanent changes
```

In [90]:
```
a
```

Out[90]:
```
array([110, 530,  65,  62,  10,  59,  79,  18,   3,  66,  81,  70,  40,
        85,  38])
```

np.delete

The numpy.delete() function returns a new array with the deletion of sub-arrays along with the mentioned axis.

In [91]:
```
a
```

Out[91]:
```
array([110, 530,  65,  62,  10,  59,  79,  18,   3,  66,  81,  70,  40,
        85,  38])
```

In [92]:
```
np.delete(a,0) # deleted 0 index item
```

Out[92]:
```
array([530,  65,  62,  10,  59,  79,  18,   3,  66,  81,  70,  40,  85,
        38])
```

In [93]:
```
np.delete(a,[0,2,4]) # deleted 0,2,4 index items
```

Out[93]:
```
array([530,  62,  59,  79,  18,   3,  66,  81,  70,  40,  85,  38])
```

Set functions

np.union1d

np.intersect1d

np.setdiff1d

np.setxor1d

np.in1d

In [94]:
```
m = np.array([1,2,3,4,5])
n = np.array([3,4,5,6,7])
```

In [95]:
```
# Union
np.union1d(m,n)
```

Out[95]:
```
array([1, 2, 3, 4, 5, 6, 7])
```

In [96]:
```
# Intersection
np.intersect1d(m,n)
```

Out[96]:
```
array([3, 4, 5])
```

In [97]:
```
# Set difference
np.setdiff1d(m,n)
```

Out[97]:
```
array([1, 2])
```

In [98]:
```
np.setdiff1d(n,m)
```

Out[98]:
```
array([6, 7])
```

In [99]:
```
# set Xor
np.setxor1d(m,n)
```

Out[99]:
```
array([1, 2, 6, 7])
```

In [100…
```
# in 1D ( like membership operator)
np.in1d(m,1)
```

Out[100…
```
array([ True, False, False, False, False])
```

In [101…
```
m[np.in1d(m,1)]
```

Out[101…
```
array([1])
```

In [102…
```
np.in1d(m,10)
```

Out[102…
```
array([False, False, False, False, False])
```

np.clip

numpy.clip() function is used to Clip (limit) the values in an array.

In [103…
```
a
```

Out[103…
```
array([110, 530,  65,  62,  10,  59,  79,  18,   3,  66,  81,  70,  40,
        85,  38])
```

In [104…
```
np.clip(a, a_min=15 , a_max =50)
```

Out[104…
```
array([50, 50, 50, 50, 15, 50, 50, 18, 15, 50, 50, 50, 40, 50, 38])
```

it clips the minimum data to 15 and replaces everything below data to 15 and maximum to 50

np.swapaxes

numpy.swapaxes() function interchange two axes of an array.

In [105…
```
arr = np.array([[1, 2, 3], [4, 5, 6]])
swapped_arr = np.swapaxes(arr, 0, 1)
```

In [106…
```
arr
```

Out[106…
```
array([[1, 2, 3],
       [4, 5, 6]])
```

In [107…
```
swapped_arr
```

Out[107…
```
array([[1, 4],
       [2, 5],
       [3, 6]])
```

In [108…
```
print("Original array:")
print(arr)
```

```
Original array:
[[1 2 3]
 [4 5 6]]
```

In [109…

```
print("Swapped array:")
print(swapped_arr)
```

```
Swapped array:
[[1 4]
 [2 5]
 [3 6]]
```

In [ ]: